Chapter 20 — Cheat Sheets and Glossary

This last chapter compresses Chapters 1–19 into pages you can scan in a minute. There is one **cheat sheet** per area of the language and platform, a **feature-by-version table** for Java 17 → 25, the build and test **commands** you type every day, an **index of the interview questions** answered in Chapter 19, a short **"where next"** guide, and an alphabetical **glossary** of 700+ terms, each linked to the section that explains it properly.

Nothing here is new. If a line doesn't make sense, follow its link back to the full explanation — and every technical term used on these sheets has a one-sentence definition in the glossary at the end (§20.19). References look like **§10.5**, meaning Chapter 10, section 5.

What you'll learn: how to use this chapter as a revision tool. Read a sheet, cover the right-hand column, and try to say the answer before you look. The glossary is a good last check before an interview: if you can't define a term in one sentence, follow its link.

Every concrete value on the sheets is tested. Primitive ranges, the `Integer` cache, the string pool, collection behaviours, stream results, `BigDecimal` and `java.time` edge cases, concurrency results and more are asserted by `code/ch20/CheatsheetChecks.java` (package `ch20` — a package is a named group of classes, like a folder), compiled with `javac --release 25` and run on JDK 26 by the book's runner (`bash tools_run_java.sh java`). A failed check throws `AssertionError`. Real output:

```
OK   20.1 syntax, types, primitives           29 checks
OK   20.2 operators & precedence              16 checks
OK   20.3 control flow & patterns             12 checks
OK   20.4 methods & arrays                    16 checks
OK   20.5 OOP                                 13 checks
OK   20.6 core APIs                           45 checks
OK   20.7 exceptions                          12 checks
OK   20.8 generics                             7 checks
OK   20.9 collections                         28 checks
OK   20.10 functional interfaces & Optional   12 checks
OK   20.11 streams & collectors               18 checks
OK   20.12 I/O & NIO.2                        11 checks
OK   20.13 concurrency                        13 checks
OK   20.14 JVM                                9 checks
OK   20.15 modern Java 17-25                  9 checks
all 250 checks passed on Java 26
```

The helper is three small methods — `eq(actual, expected, label)`, `yes(condition, label)` and `throwsA(ExceptionType.class, code, label)` — because Java's own `assert` keyword is switched off unless you run `java -ea` (§19.1). A few lines as a taste:

```
Integer i1 = 127, i2 = 127, i3 = 128, i4 = 128;
eq(i1 == i2, true, "Integer cache -128..127");
eq(i3 == i4, false, "outside the cache: different objects");
throwsA(IllegalStateException.class, () -> emps.stream().collect(Collectors.toMap(Emp::dept, Emp::name)),
        "toMap duplicate key throws");
eq(LocalDate.of(2024, 1, 31).plusMonths(1), LocalDate.of(2024, 2, 29), "plusMonths clamps to month end");
```

What's inside

§	Sheet	Built from
20.1	Syntax, types and primitive ranges	Ch 1, 2
20.2	Operators and precedence	Ch 2
20.3	Control flow, `switch` and patterns	Ch 3
20.4	Methods and arrays	Ch 4
20.5	OOP: classes, interfaces, records, enums, sealed types	Ch 5, 6
20.6	Core APIs: strings, numbers, dates	Ch 7
20.7	Exceptions	Ch 8
20.8	Generics and PECS	Ch 9
20.9	Collections: big-O and which to choose	Ch 10
20.10	Functional interfaces, lambdas and `Optional`	Ch 11
20.11	Streams and collectors	Ch 12
20.12	I/O and NIO.2	Ch 13
20.13	Concurrency	Ch 14, 15
20.14	The JVM: memory, GC, flags, tools	Ch 1, 16
20.15	Modern Java by version, 17 → 25	Ch 17
20.16	Testing, build and tooling commands	Ch 1, 18
20.17	Interview question index	Ch 19
20.18	Where next	Ch 18
20.19	Glossary A–Z	all

20.1 Syntax, types and primitive ranges

Program shapes (§1.5, §1.10, §4.9):

```
// Classic: file Hello.java, class name = file name
public class Hello {
    public static void main(String[] args) { System.out.println("Hi " + args.length); }
}

// Java 25 compact source file (JEP 512): no class declaration, instance main, java.lang.IO
void main() { IO.println("Hi"); }
```

Command	Does	Since
`javac Hello.java` → `java Hello`	compile to `Hello.class`, then run it on the JVM	1.0
`javac -d out $(find src -name '*.java')` → `java -cp out pkg.Main`	compile a tree into `out/`, run with a class path (the folders and JARs searched for classes)	—
`java Hello.java`	compile in memory and run a single source file	11
`java Main.java` (other files next to it)	launch a multi-file source program	22
`jshell`	interactive REPL (read-eval-print loop): type expressions, see results	9
`javap -c Hello`	disassemble bytecode (list the JVM instructions in a `.class` file)	—

Basic rules (§2.1): statements end with `;`; blocks use `{}`; Java is case-sensitive; identifiers (the names you give variables, methods and classes) start with a letter, `_` or `$` (a lone `_` is not a name — since 22 it means "unnamed"); comments are `//`, `/* */` and `/** Javadoc */` (documentation comments turned into HTML API docs) (Markdown `///` doc comments since 23). Naming: `ClassName`, `methodAndVariable`, `CONSTANT_NAME`, `com.example.pkg`. Contextual keywords — `var`, `record`, `sealed`, `non-sealed`, `permits`, `yield`, `when`, `module` — are keywords only in their positions.

The eight primitives (§2.3) — everything else is a reference type (§2.5):

Type	Bits	Range	Default (fields)	Literal
`byte`	8	−128 … 127	`0`	`(byte) 1`
`short`	16	−32 768 … 32 767	`0`	`(short) 1`
`int`	32	−2 147 483 648 … 2 147 483 647 (±2.1 × 10^9)	`0`	`42`, `0x2A`, `0b101010`, `052`, `1_000_000`
`long`	64	−9 223 372 036 854 775 808 … 9 223 372 036 854 775 807 (±9.2 × 10^{18})	`0L`	`42L`
`float`	32	±3.4028235 × 10^{38}, ~7 significant digits	`0.0f`	`3.14f`
`double`	64	±1.7976931348623157 × 10^{308}, ~15–16 digits	`0.0`	`3.14`, `1e-9`
`char`	16	0 … 65 535 (unsigned UTF-16 code unit: one 16-bit piece of text)	`'\u0000'`	`'A'`, `'\n'`, `'\u0041'`
`boolean`	—	`true`, `false`	`false`	`true`

- `Double.MIN_VALUE` (4.9E-324) is the smallest **positive** double; the most negative is `-Double.MAX_VALUE`.
- Fields get defaults (`0`, `false`, `null`); **local variables don't** — reading one before assigning it is a compile error (§2.2).
- Money: `long` cents or `BigDecimal` (an exact decimal-number class), never `double` (§2.11).

Conversions (§2.7):

Rule	Example	Result
Widening is automatic: `byte → short → int → long → float → double`, `char → int`	`long l = 42;`	ok
Narrowing needs a cast and keeps the low bits	`(byte) 200`	`-56`
Casting a floating value to an integer truncates toward zero	`(int) 3.99`	`3`
Arithmetic promotes `byte`/`short`/`char` to `int`	`byte b = 10; b = b + 1;`	compile error
Compound assignment hides a cast	`byte b = 10; b += 300;`	`54`
`char` is a number	`(int) 'A'`, `(char) ('a' + 1)`	`65`, `'b'`

Integer and floating-point facts (§2.10, §2.11):

Expression	Value	Why
`Integer.MAX_VALUE + 1`	`-2147483648`	silent wrap-around; use `Math.addExact` to get an `ArithmeticException`
`7 / 2`, `-7 / 2`	`3`, `-3`	integer division truncates toward zero
`-7 % 3` / `Math.floorMod(-7, 3)`	`-1` / `2`	`%` takes the dividend's sign
`7 / 2.0`	`3.5`	one `double` operand → `double` arithmetic
`0.1 + 0.2`	`0.30000000000000004`	binary fractions can't represent 0.1 exactly
`1.0 / 0`, `0.0 / 0`, `1 / 0`	`Infinity`, `NaN` (Not a Number), `ArithmeticException`	floats have special values; ints throw
`Double.NaN == Double.NaN`	`false`	test with `Double.isNaN`

`var` (Java 10, §2.6): local variables with an initializer only (also in `for` loops and lambda parameters); not for fields, parameters or return types; `var x = null;` and `var a = {1, 2};` don't compile. The type is fixed at compile time — `var` is not dynamic typing (types decided while the program runs, as in Python).

Text blocks (Java 15, §2.12): `"""` + newline opens; common indentation is stripped (the closing `"""` position counts); a closing delimiter on its own line adds a trailing `\n`; `\` at line end joins lines; `\s` keeps a trailing space.

Console I/O (§2.13, §13.8): `System.out.println/print/printf`; input with `Scanner(System.in)` (`nextInt`, `nextLine` — beware the leftover newline), `BufferedReader`, or `IO.readln("prompt")` in Java 25. `System.console()` returns `null` without an interactive terminal (IDEs, pipes) — always check it (Java 22–24 briefly returned a JLine-based console (JLine is a library for rich terminal input) even then; 25 reverted).

20.2 Operators and precedence

From highest to lowest (§2.8, §2.9). Operands are always evaluated **left to right**; associativity only decides grouping.

Level	Operators	Associativity
1 Postfix	`x++` `x--`	left
2 Unary	`++x` `--x` `+` `-` `~` `!` `(type)`	right
3 Multiplicative	`*` `/` `%`	left
4 Additive	`+` `-` (and `String` `+`)	left
5 Shift	`<<` `>>` `>>>`	left
6 Relational	`<` `>` `<=` `>=` `instanceof`	left
7 Equality	`==` `!=`	left
8–10 Bitwise	`&` then `^` then `|`	left
11–12 Logical	`&&` then `||` (short-circuit: stop once the answer is known)	left
13 Ternary	`? :`	right
14 Assignment, lambda	`=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=` `>>>=` `->`	right

Memory hook: **unary → multiply → add → shift → compare → equal → bitwise → logical → ternary → assign.**

Expression	Value	Lesson
`2 + 3 * 4`	`14`	`*` before `+`
`10 - 4 - 3`	`3`	left-associative
`1 + 2 + "3" + 4 + 5`	`"3345"`	numbers add until the first `String`, then everything concatenates
`true || false && false`	`true`	`&&` binds tighter than `||`
`x & 1 == 0`	compile error for `int x`	`==` binds tighter than `&` — write `(x & 1) == 0`
`i = 5; i++ + ++i`	`12`	`5 + 7`; never write this in real code
`-8 >> 1`, `-8 >>> 28`	`-4`, `15`	`>>` keeps the sign, `>>>` fills with zeros
`5 ^ 3`, `~5`, `1 << 10`	`6`, `-6`, `1024`	XOR, complement (−(n+1)), shift = ×2ⁿ
`a = b = c = 7`	all `7`	assignment is right-associative
`s != null && s.length() > 0` with `s = null`	`false`, no exception	`&&` stops at the first `false`
`o instanceof String str && str.length() == 4`	`true` for `"text"`	pattern variable in scope after `&&` (Java 16)

`==` on objects compares **references**; use `equals` for content (§6.13, §7.6).

20.3 Control flow, `switch` and patterns

Branches and loops (§3.1, §3.8, §3.9):

Construct	Shape	Notes
`if` / `else if` / `else`	`if (cond) { … } else { … }`	condition must be `boolean` — `if (1)` doesn't compile
Ternary	`cond ? a : b`	an expression; keep it short
`for`	`for (int i = 0; i < n; i++)`	index loops
Enhanced `for`	`for (String s : list)`	arrays and any `Iterable`; can't remove safely inside
`while` / `do … while`	test first / body runs at least once	
`break` / `continue`	leave / skip to next iteration of the innermost loop	`break outer;` / `continue outer;` with a label

The four `switch` shapes (§3.2, §3.3, §3.4):

Shape	Falls through (runs on into the next case)?	Produces a value?	Must be exhaustive?
Statement, `case X:`	**yes** — until `break`	no	no (unless it uses patterns or `null`)
Statement, `case X ->` (14)	no	no	no (same exception)
Expression, `case X ->` (14)	no	yes; blocks use `yield`	**yes**
Expression, `case X:` + `yield`	yes	yes	yes

```
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "weekday";
    case 6, 7 -> "weekend";
    default -> { yield "invalid"; }
};
```

Selector types (the type of the value being switched on): `int` and smaller (`char`, `byte`, `short`) and their wrappers, `String`, enums — and, with patterns (21), **any reference type**. `long`, `float`, `double` and `boolean` selectors need *primitive patterns*, which are still **preview** (shipped but switched off by default, and may still change) in Java 25.

Pattern matching (§3.5, §3.6, §3.7):

Feature	Syntax	Final in
Type pattern in `instanceof`	`if (o instanceof String s && !s.isEmpty())`	16
Type pattern in `switch`	`case Integer i ->`	21
Guard	`case Integer i when i < 0 ->`	21
`null` label	`case null ->` or `case null, default ->`	21
Record pattern (nestable)	`case Point(int x, int y) when x == y ->`	21
`var` in a record pattern	`case Point(var x, var y) ->`	21
Unnamed pattern / variable	`case Point(var x, _) ->`, `catch (Exception _)`, `for (var _ : list)`	22

Rules to remember: without `case null`, a `null` selector throws `NullPointerException`; the first matching label wins, and a label that can never match (dominated, e.g. `CharSequence` before `String`) is a compile error; a pattern `switch` must be exhaustive — add `default` or cover every permitted subtype of a **sealed** type:

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}

static double area(Shape s) {
    return switch (s) {                // no default needed: the compiler knows all subtypes
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.side() * q.side();
    };
}
```

Loop pitfalls (§3.10): off-by-one (`<=` vs `<`), modifying a collection inside for-each (→ `ConcurrentModificationException`), `float` loop counters, `==` on strings in conditions, and infinite `while` loops whose variable never changes.

20.4 Methods and arrays

Methods (§4.1–§4.5):

Topic	Rule
Signature	name + parameter types (not the return type, not parameter names)
Passing	**always by value**; for objects the value is a copy of the reference — the callee can mutate the object but not rebind the caller's variable
Overload resolution	phase 1: exact match or widening; phase 2: boxing/unboxing; phase 3: varargs. `over(5)` with `over(long)`, `over(Integer)`, `over(int...)` picks `long`
Varargs	`int... nums` is an `int[]`; must be the last parameter; one per method; zero arguments allowed
`return`	a non-`void` method must return on every path
Recursion	needs a base case; each call adds a stack frame → `StackOverflowError` if too deep (Java has no tail-call elimination — reusing the current frame when a method's last act is to call itself)
`static`	called on the class, has no `this` (§5.5)

Arrays (§4.6, §4.7):

```
int[] a = new int[3];             // [0, 0, 0] — elements get default values
int[] b = {3, 1, 2};              // initializer
String[] names = new String[2];   // [null, null]
int[][] grid = new int[3][4];     // rectangular
int[][] jagged = {{1}, {2, 3}, {}};  // rows of different lengths
a.length                           // a field, not a method; fixed forever
```

Index `0 … length − 1`; anything else → `ArrayIndexOutOfBoundsException`. Arrays are objects: `a.equals(b)` and `==` compare identity; `a.toString()` prints something like `[I@1b6d3586`.

`java.util.Arrays` (§4.8):

Method	Does	Example result
`toString(a)` / `deepToString(m)`	print 1D / nested	`[1, 2, 3]`, `[[1], [2, 3], []]`
`equals(a, b)` / `deepEquals`	compare contents	`true` for `{3,1,2}` vs `{3,1,2}`
`sort(a)`, `sort(a, from, to)`, `sort(T[], cmp)`	sort in place (dual-pivot quicksort for primitives, stable merge sort — TimSort — for objects)	
`binarySearch(a, key)`	on a **sorted** array; not found → `-(insertionPoint) − 1`	`binarySearch([1,2,3], 5)` = `-4`
`copyOf(a, n)`, `copyOfRange(a, from, to)`	new array, padded with defaults	`copyOf([1,2,3], 5)` = `[1, 2, 3, 0, 0]`
`fill(a, v)`, `setAll(a, i -> …)`	set every element	
`asList(…)`	fixed-size `List` **view** of the array — `add` throws, `set` writes through	
`stream(a)`	`IntStream` / `Stream<T>`	
`mismatch(a, b)`, `compare(a, b)`	first differing index / lexicographic (dictionary) order (9)	

Command-line arguments: `args` is never `null` (empty array when none); all elements are `String`s; the program name is not included (§4.9).

20.5 OOP: classes, interfaces, records, enums and sealed types

Access modifiers (§5.6):

Modifier	Same class	Same package	Subclass (other package)	Everywhere
`private`	✓			
(none) package-private	✓	✓		
`protected`	✓	✓	✓ (through inheritance)	
`public`	✓	✓	✓	✓

Top-level classes can only be `public` or package-private; one `public` top-level class per file, named like the file.

Constructors and initialisation (§5.3, §5.10):

- If you write no constructor, the compiler adds a no-argument one; write any constructor and it doesn't.
- `this(...)` calls another constructor of the same class, `super(...)` the parent's; if you write neither, `super()` is inserted. Since **Java 25** (JEP 513 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature) statements that don't use `this` — validation, computing arguments, assigning the class's own fields — may appear **before** `super(...)`/`this(...)`.
- Order: static fields and `static {}` blocks run **once**, in source order, when the class is initialised → for each `new`: parent constructor → instance field initialisers and `{}` blocks in source order → the rest of the constructor body.

`static` and `final` (§5.5, §5.8): `static` = one per class, shared; a static method has no `this`. `final` variable = assigned once; `final` method = can't be overridden; `final` class = can't be extended. `final` on a reference doesn't freeze the object.

Inheritance and overriding (§6.1–§6.4):

Overriding rule	Detail
Same signature	otherwise it's an overload; use `@Override` to let the compiler check
Return type	same or a subtype (covariant)
Access	same or wider (`protected` → `public` ok, not the reverse)
Checked exceptions	same, narrower or none — never new or broader
Not overridable	`private`, `static` (hidden instead), `final` methods; constructors
Dispatch	instance methods use the object's run-time class; fields and static methods use the compile-time type

```
class Base    { static String who() { return "Base"; }    String me() { return "Base"; }
                String call() { return me() + "/" + who(); } }
class Derived extends Base { static String who() { return "Derived"; }
                @Override String me() { return "Derived"; } }
new Derived().call()   // "Derived/Base"
```

Choosing a type (§6.12):

Need	Use	Key facts
Mutable object with identity and lifecycle	**`class`** (`final` unless designed for extension)	full control of encapsulation (hiding state behind methods)
Shared state/code for a family of related classes	**`abstract class`**	fields, constructors, any access; single inheritance; can't be instantiated (§6.5)
A capability many unrelated types can have	**`interface`**	many per class; `abstract`, `default` (8), `static` (8), `private` (9) methods; fields are `public static final` constants; no instance state (§6.6)
Immutable data where equal values are interchangeable	**`record`** (16)	private final fields, canonical constructor, accessors `x()`, `equals`/`hashCode`/`toString`; implicitly `final`; extends `Record`; no extra instance fields; compact constructor for validation (§6.8)
A fixed set of instances known at compile time	**`enum`**	fields, constructors (private), methods, per-constant bodies; `values()`, `valueOf`, `ordinal()`; `EnumSet`/`EnumMap` (§6.9)
A closed set of variants	**`sealed`** interface/class (17) + records	`permits` lists subtypes; each is `final`, `sealed` or `non-sealed`; exhaustive `switch` without `default` (§6.7)

```
record Range(int lo, int hi) {
    Range {                                          // compact constructor
        if (lo > hi) throw new IllegalArgumentException(lo + " > " + hi);
    }
}
new Point(1, 2).equals(new Point(1, 2))          // record Point(int x, int y): true;  toString() = "Point[x=1, y=2]"
```

Nested types (§6.10, §6.11):

Kind	Declared	Has an outer instance?	Typical use
Static nested class	`static class Node` inside a class	no	helpers, builders, `Map.Entry`-like types — the default choice
Inner class	`class Itr` inside a class	yes (`Outer.this`)	iterators that read the outer object's fields
Local class	inside a method body	yes if in an instance method	one-off helper with a name
Anonymous class	`new Interface() { … }`	yes if in an instance method	one-off implementation with state or several methods
Lambda	`x -> …`	captures effectively final locals	single-method (functional) interfaces

Nested records, enums and interfaces are implicitly `static`.

`Object` methods contract (§6.13): `equals` must be reflexive, symmetric, transitive, consistent, and `false` for `null`; equal objects **must** have equal `hashCode`s; override both or neither (`Objects.equals`, `Objects.hash` help). `toString` defaults to `ClassName@hexHash`. Immutability recipe (§5.9): `final` class, `private final` fields, no setters, defensive copies in and out (`List.copyOf`), don't leak `this` from the constructor. Design-level treatment: LLD handbook.

20.6 Core APIs: String, numbers, dates

Strings (§7.1, §7.2) are immutable; every "modifying" method returns a new `String`.

Fact	Value
`"java" == "java"` (two literals)	`true` — both are the pooled object
`"java" == new String("java")`	`false` — `new` always makes a new object
`"java" == new String("java").intern()`	`true`
`"java" == part + "va"` (run time, `part = "ja"`)	`false`; `"ja" + "va"` (constants) is folded → `true`
`"Java".substring(1, 3)`	`"av"` — end index exclusive
`"a,b,,".split(",").length` / `split(",", -1)`	`2` / `4` — trailing empties dropped unless limit is negative
`"a.b".replace(".", "-")` / `replaceAll(".", "-")`	`"a-b"` / `"---"` — `replaceAll` takes a **regex**
`" ".isBlank()` / `" ".isEmpty()`	`true` / `false`
`compareTo`	negative, zero or positive — not necessarily −1/1

Everyday methods: `length`, `charAt`, `indexOf` (−1 if absent), `contains`, `startsWith`/`endsWith`, `substring`, `strip` (Unicode-aware; prefer to `trim`), `toLowerCase(Locale.ROOT)` (a `Locale` is a language/region setting), `equalsIgnoreCase`, `split`, `String.join`, `repeat` (11), `lines` (11), `formatted` (15), `chars`, `codePoints`.

Builders (§7.3): `StringBuilder` (mutable, not synchronised — use it for loops: `append`, `insert`, `reverse`, `deleteCharAt`, `setLength(0)`); `StringBuffer` (synchronised, legacy); `StringJoiner` / `String.join` / `Collectors.joining` for delimited lists. `s += x` in a loop is O(n²).

Formatting (§7.4): `String.format(Locale.US, "%5.2f|%-4s|%04d|%,d", 3.14159, "ab", 42, 1234567)` → `" 3.14|ab  |0042|1,234,567"`. Conversions `%s %d %f %e %x %c %b %n %%`; flags `-` left, `0` pad, `,` grouping, `+` sign; `%2$s` argument index. Always pass a `Locale` when the output is parsed by machines.

`Math` (§7.5):

Call	Result	Note
`Math.round(2.5)`, `Math.round(-2.5)`	`3`, `-2`	ties toward +∞; returns `long` for `double`
`Math.rint(2.5)`	`2.0`	ties to even
`Math.floorDiv(-7, 2)`, `floorMod(-7, 3)`	`-4`, `2`	round toward −∞
`Math.abs(Integer.MIN_VALUE)`	`-2147483648`	overflow! `absExact` throws
`Math.clamp(15, 0, 10)`	`10`	Java 21
`addExact`, `multiplyExact`, `toIntExact`, `powExact` (25)	throw `ArithmeticException` on overflow	
`Math.random()`	`double` in [0.0, 1.0)	prefer `RandomGenerator`

Wrappers and autoboxing (§7.6):

Fact	Detail
`Integer a = 127, b = 127; a == b`	`true` — `Integer.valueOf` caches **`−128 … 127`** (also `Short`, `Byte`, `Long`, `Character` ≤ 127, both `Boolean`s)
`Integer c = 128, d = 128; c == d`	`false` — always compare wrappers with `equals`
`int x = (Integer) null`	`NullPointerException` on unboxing
`list.remove(1)` vs `list.remove(Integer.valueOf(1))`	removes **index** 1 vs the **value** 1
`Integer.parseInt("ff", 16)`	`255`; bad input → `NumberFormatException`
`Boolean.parseBoolean("YES")`	`false` — only `"true"` (any case) is true

`BigInteger` / **`BigDecimal`** (§7.7): immutable, arbitrary precision (as many digits as needed).

Rule	Example
Build from a `String` (or `BigDecimal.valueOf(double)`)	`new BigDecimal("0.1").add(new BigDecimal("0.2"))` = `0.3`; `new BigDecimal(0.1)` = `0.1000000000000000055511…`
`equals` checks value **and scale** (digits after the point); `compareTo` checks value only	`2.0` vs `2.00`: `equals` false, `compareTo` 0 — so `HashSet<BigDecimal>` can hold both
Non-terminating division needs a scale or `MathContext`	`ONE.divide(valueOf(3))` → `ArithmeticException`; `divide(valueOf(3), 4, HALF_UP)` = `0.3333`
Rounding modes	`2.345` → `HALF_UP` `2.35`, `HALF_EVEN` (banker's) `2.34`
`BigInteger`	`BigInteger.TWO.pow(100)` has 31 digits

`java.time` (§7.8–§7.11) — immutable, thread-safe (safe to share between threads); months are 1–12:

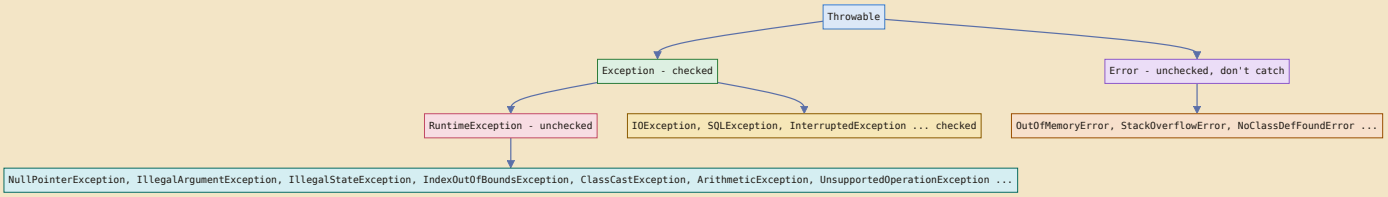
Class	Represents	Example
`LocalDate` / `LocalTime` / `LocalDateTime`	wall-calendar date/time, **no zone**	birthdays, opening hours
`ZonedDateTime`	date-time in a region (`ZoneId.of("Europe/Paris")`), DST-aware (handles daylight-saving clock changes)	meetings
`OffsetDateTime`	date-time with a fixed offset (`+02:00`)	API timestamps
`Instant`	a point on the UTC (world reference time) time-line	logs, DB timestamps
`Duration`	seconds + nanos (machine time)	`Duration.ofMinutes(90)` → `PT1H30M`
`Period`	years, months, days (calendar)	2024-01-15 → 2025-03-20 = `P1Y2M5D`
`DateTimeFormatter`	format/parse, thread-safe	`ofPattern("yyyy-MM-dd HH:mm")`

Edge cases (all checked): `LocalDate.of(2024, 1, 31).plusMonths(1)` → `2024-02-29` (clamped); in `America/New_York`, 2025-03-09 01:30 plus 1 hour → 03:30, and that day lasts 23 hours; `yyyy` vs `YYYY` for 2025-12-29 → `2025` vs `2026` (week-based year — a classic bug); `MM` month vs `mm` minute; `HH` 0–23 vs `hh` 1–12; `LocalDate.of(2025, 2, 29)` → `DateTimeException`. Legacy `Date`/`Calendar`/`SimpleDateFormat` are mutable and not thread-safe — convert at the edges.

Random numbers (§7.12): `new Random(42)` is reproducible (same seed → same sequence); `ThreadLocalRandom.current()` for concurrent code; `SecureRandom` for tokens and passwords; `RandomGenerator.of("L64X128MixRandom")` (17) for choosing an algorithm; `random.nextInt(lo, hi)` (17) excludes `hi`.

20.7 Exceptions

The hierarchy (§8.2):



Kind	Which classes	Compiler rule	Meaning
Checked	`Exception` and subclasses **except** the `RuntimeException` branch	must be caught or declared with `throws` (§8.3)	a recoverable problem outside the program's control (file, network, DB)
Unchecked	`RuntimeException` and subclasses	no rule	a programming error (bad argument, bad state, `null`)
Error	`Error` and subclasses (also unchecked)	no rule	the JVM or environment is in trouble — let it propagate

Common exception	Thrown when	Kind
`NullPointerException`	dereferencing `null` (using it to reach a field or method; helpful messages name the `null` part: Java 14, on by default since 15)	unchecked
`IllegalArgumentException` / `NumberFormatException`	bad argument / unparsable number	unchecked
`IllegalStateException`	object not in the right state (stream reused, `toMap` duplicate key)	unchecked
`ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException`	bad index	unchecked
`ClassCastException`, `ArithmeticException`	bad cast / integer `/ 0`, overflow in `*Exact`	unchecked
`UnsupportedOperationException`	mutating an unmodifiable collection	unchecked
`ConcurrentModificationException`	structural change during iteration	unchecked
`IOException`, `FileNotFoundException`, `NoSuchFileException`	I/O failure	**checked**
`InterruptedException`	a blocked thread was interrupted	**checked**
`SQLException`	JDBC (Java's database API) failure	**checked**
`UncheckedIOException`	an `IOException` wrapped for lambdas/streams	unchecked

Syntax rules (§8.4–§8.6):

Construct	Rule
`try { } catch (A e) { } finally { }`	`catch` or `finally` (or both) required; `finally` runs on normal exit, exception, `return`, `break`
Catch order	subclasses before superclasses, else "exception has already been caught" compile error
Multi-catch `catch (A | B e)`	`A` and `B` must not be subtype-related; `e` is effectively final
`return` in `finally`	overrides the `try`'s return **and swallows exceptions** — never do it (`try { return 1; } finally { return 2; }` returns `2`)
try-with-resources `try (var r = open())`	any `AutoCloseable`; closed automatically in **reverse** order; a close failure is added as **suppressed** to the body's exception (`getSuppressed()`)
`finally` doesn't run	only if the JVM exits (`System.exit`, crash) or the thread never leaves `try`
Rethrow	`catch (Exception e) { throw e; }` — precise rethrow keeps the narrower checked types
Chaining	`throw new ServiceException("load failed", e)` — keep the cause (§8.8)

```
try (Res a = new Res("A", log); Res b = new Res("B", log)) {
    throw new RuntimeException("body failed");
}   // log: open A, open B, body, close B, close A — both close() failures become suppressed
```

Custom exceptions (§8.7): extend `Exception` (checked) or `RuntimeException` (unchecked); provide `(String message)` and `(String message, Throwable cause)` constructors; add fields for context.

Best practices (§8.10): throw early (validate arguments with `Objects.requireNonNull`, `IllegalArgumentException`), catch late (where you can really handle it); never swallow (`catch (Exception e) {}`); don't use exceptions for control flow; catch the most specific type; translate low-level exceptions at layer boundaries (where, say, database code hands over to business code) and keep the cause; log **or** rethrow, not both; clean up with try-with-resources. Reading traces: the first line is the type and message, then frames newest-first; read `Caused by:` sections bottom-up to find the root (§8.9).

20.8 Generics and PECS

Syntax (§9.1–§9.5):

Form	Example
Generic class / record	`class Box<T> { T value; }`, `record Pair<A, B>(A first, B second) {}`
Generic interface	`interface Repository<T, ID> { Optional<T> findById(ID id); }`
Generic method	`static <T> List<T> repeat(T x, int n)` — type parameter before the return type
Explicit type argument	`Collections.<String>emptyList()` (rarely needed — inference)
Diamond	`Map<String, List<Integer>> m = new HashMap<>();`
Bounded type	`<T extends Number>`, several bounds: `<T extends Number & Comparable<T>>` (class first)
Recursive bound	`<T extends Comparable<? super T>> T max(List<? extends T> xs)`
Wildcards	`List<?>`, `List<? extends Number>`, `List<? super Integer>`

Naming convention: `T` type, `E` element, `K`/`V` key/value, `R` result, `ID` identifier.

Invariance and PECS (§9.6): `List<Integer>` is **not** a `List<Number>` (arrays are covariant — a `String[]` counts as an `Object[]` — and fail at run time with `ArrayStoreException`; generics refuse at compile time).

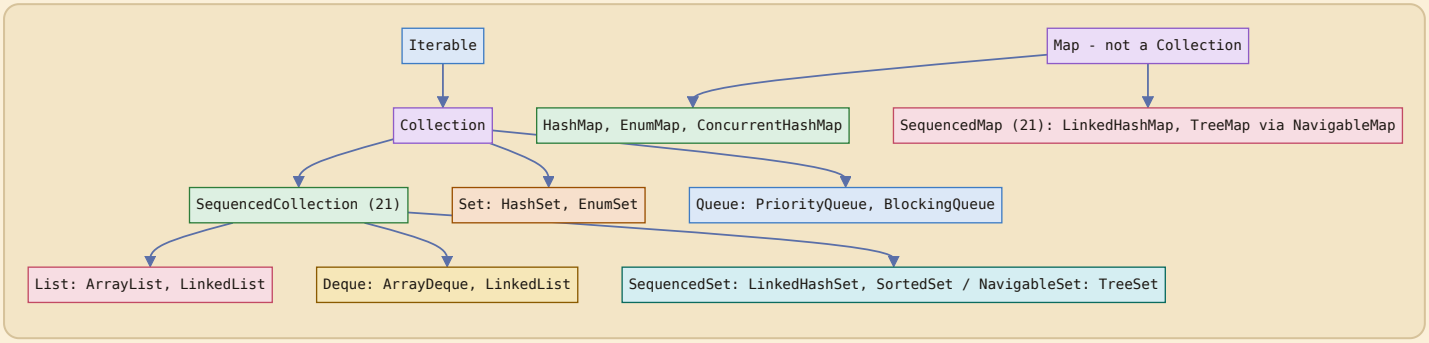
Wildcard	You can read as	You can add	Use when the parameter…	Example
`List<? extends Number>`	`Number`	nothing (except `null`)	**produces** values for you	`double sumAll(List<? extends Number> xs)`
`List<? super Integer>`	`Object`	`Integer`	**consumes** values you give it	`void fill(List<? super Integer> sink)` accepts `List<Number>`, `List<Object>`
`List<?>`	`Object`	nothing	you only use `Object` methods / size	`printAll(List<?> xs)`
`List<T>`	`T`	`T`	you both read and write	`swap(List<T> xs, int i, int j)`

Producer **E**xtends, **C**onsumer **S**uper — e.g. `Collections.copy(List<? super T> dest, List<? extends T> src)`. Don't use wildcards in return types.

Type erasure (§9.7, §9.8): generics are checked by the compiler, then erased to their bounds (`Object` if unbounded) with casts inserted. `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()` is `true`. So you **can't**: use primitives (`List<int>`), `new T()`, `new T[n]`, `instanceof List<String>`, use `T` in static members, overload `f(List<String>)` with `f(List<Integer>)` (same erasure), or make a generic `Throwable`. Workarounds: `Supplier<T>` factories, `Class<T>` tokens, `List<T>` instead of `T[]`. Raw types (`List raw`) switch checking off — they exist only for pre-Java-5 code. Eliminate unchecked warnings; suppress only on the smallest scope with a comment; mark safe generic varargs methods `@SafeVarargs`.

20.9 Collections: big-O and which to choose

The hierarchy (§10.1):



Big-O (§10.12) — `*` amortised (averaged over many calls), `†` average (worst case O(log n) in a tree-ified bucket):

Collection	add at end	add/remove at front	get(i)	contains / get(key)	remove element	Order	null ?
`ArrayList`	O(1)*	O(n)	O(1)	O(n)	O(n)	index	yes
`LinkedList`	O(1)	O(1)	O(n)	O(n)	O(n)	index	yes
`ArrayDeque`	O(1)*	O(1)*	–	O(n)	O(n)	head → tail	**no**
`PriorityQueue`	O(log n)	`poll` O(log n), `peek` O(1)	–	O(n)	O(n)	heap (iteration unsorted)	no
`HashSet` / `HashMap`	O(1)†	–	–	O(1)†	O(1)†	none	one `null` key
`LinkedHashSet` / `LinkedHashMap`	O(1)†	–	–	O(1)†	O(1)†	insertion (or access)	one `null` key
`TreeSet` / `TreeMap`	O(log n)	–	–	O(log n)	O(log n)	sorted	no `null` keys (natural order)
`EnumSet` / `EnumMap`	O(1)	–	–	O(1)	O(1)	enum order	no
`CopyOnWriteArrayList`	O(n)	O(n)	O(1)	O(n)	O(n)	index	yes
`ConcurrentHashMap`	O(1)†	–	–	O(1)†	O(1)†	none	**no**

Which one? (§10.13) Key → value: `HashMap`; insertion order or LRU (least-recently-used eviction): `LinkedHashMap`; sorted or range queries: `TreeMap`. Unique elements: `HashSet` / `LinkedHashSet` / `TreeSet` by the same rule. Indexed or append: `ArrayList` (the default list). Stack or queue: `ArrayDeque` (not `Stack`, not `LinkedList`). "Smallest next": `PriorityQueue`. Enum keys: `EnumMap` / `EnumSet`. Fixed data: `List.of` / `Set.of` / `Map.of`. Shared between threads: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`.

`HashMap` internals (§10.5): table of buckets, default capacity 16, load factor 0.75 (resize — double — when size > capacity × 0.75); index = `(h ^ (h >>> 16)) & (capacity − 1)`; collisions chain in a list that turns into a red-black tree (a self-balancing sorted tree) past 8 entries once the table has ≥ 64 buckets; lookup = hash, then `equals`. Keys must have consistent `equals` / `hashCode` and must not change while in the map.

Unmodifiable vs views (§10.10):

Factory	Changes through it?	Sees later changes to the source?	null s
`List.of`, `Set.of`, `Map.of` (9)	no — `UnsupportedOperationException`	— (no source)	**rejected** (NPE); `Set.of("a","a")` → `IllegalArgumentException`
`List.copyOf` (10), `stream.toList()` (16)	no	no — a snapshot	`copyOf` rejects; `toList()` **allows** `null` (unlike `Collectors.toUnmodifiableList()`)
`Collections.unmodifiableList(src)`	no	**yes** — a read-only view	allowed
`Arrays.asList(array)`	`set` yes (writes through to the array); `add` / `remove` no	yes	allowed

API you use daily (§10.2–§10.6, §10.9, §10.11):

Want	Call	Result in the checks
Count words	`map.merge(w, 1, Integer::sum)`	`{a=1, b=3, c=1}` in a `TreeMap`
Group into lists	`map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)`	
Default	`map.getOrDefault(k, -1)`	`-1`
Previous value	`map.put("a", 2)`	returns the old value (`null` if none)
Nearest keys	`treeMap.floorKey(25)`, `ceilingKey(25)` on {10, 20, 30}	`20`, `30`; `floorKey(5)` = `null`
Sub-ranges	`headMap(20)` / `tailMap(20)`	`[10]` / `[20, 30]` (head exclusive, tail inclusive)
Stack	`deque.push(x)`, `pop()`, `peek()`	`push` = `addFirst`
Queue	`deque.offer(x)`, `poll()`, `peek()`	`offer` = `addLast`; `poll` returns `null` when empty, `remove` throws
Min-heap / max-heap	`new PriorityQueue<>()` / `new PriorityQueue<>(Comparator.reverseOrder())`	`poll()` of {5, 1, 4}: `1` / `5`
LRU cache	`new LinkedHashMap<>(16, 0.75f, true)` + override `removeEldestEntry` → `size() > cap`	
Ends of an ordered collection (21)	`getFirst()`, `getLast()`, `addFirst`, `removeLast`, `reversed()`	`[x, y, z]` → `x`, `z`, `[z, y, x]`
Sort with several keys	`list.sort(comparing(String::length).thenComparing(reverseOrder()))`	`[fig, pear, kiwi, apple]`
Remove while iterating	`removeIf(pred)` or `iterator.remove()`	for-each + `list.remove` → `ConcurrentModificationException`
Utilities	`Collections.sort`, `reverse`, `shuffle`, `frequency`, `max`, `nCopies`, `emptyList`	

Ordering (§10.8): `Comparable<T>.compareTo` = the class's one natural order; `Comparator<T>` = any number of external orders (`comparing`, `thenComparing`, `reversed`, `nullsFirst`). Never compare by subtraction (`a - b` overflows) — use `Integer.compare`. `TreeSet`/`TreeMap` treat `compareTo == 0` as "equal", so keep it consistent with `equals`.

20.10 Functional interfaces, lambdas and `Optional`

`java.util.function` (§11.2, §11.3) — a *functional interface* has exactly one abstract method:

Interface	Shape	Method	Typical use
`Supplier<T>`	`() → T`	`get`	factories, lazy defaults, `orElseGet`
`Consumer<T>` / `BiConsumer<T,U>`	`T → void` / `(T,U) → void`	`accept`	`forEach`, `Map.forEach`
`Predicate<T>` / `BiPredicate<T,U>`	`T → boolean`	`test`	`filter`, `removeIf`
`Function<T,R>` / `BiFunction<T,U,R>`	`T → R` / `(T,U) → R`	`apply`	`map`, `computeIfAbsent`, `merge`
`UnaryOperator<T>` / `BinaryOperator<T>`	`T → T` / `(T,T) → T`	`apply`	`replaceAll`, `reduce`
`IntPredicate`, `IntFunction<R>`, `ToIntFunction<T>`, `IntUnaryOperator`, `IntBinaryOperator`, `IntSupplier`, `ObjIntConsumer<T>` …	primitive versions (also `Long…`, `Double…`)	`test`, `apply`, `applyAsInt`, `getAsInt` …	avoid boxing in hot code (code that runs very often)
`Runnable` / `Callable<V>`	`() → void` / `() → V`, may throw	`run` / `call`	tasks
`Comparator<T>`	`(T,T) → int`	`compare`	sorting

Lambda syntax: `() -> 42`, `x -> x * 2`, `(a, b) -> a + b`, `(String s) -> s.length()`, `(var x, var y) -> …`, block form `x -> { …; return r; }`. Captured local variables must be **effectively final** (assigned once) (§11.5); fields can change. `this` in a lambda is the enclosing object (in an anonymous class it's the anonymous object).

Method references (§11.4):

Kind	Syntax	Equivalent lambda
Static method	`Integer::sum`	`(a, b) -> Integer.sum(a, b)`
Bound instance (specific object)	`System.out::println`, `"*"::repeat`	`x -> System.out.println(x)`
Unbound instance (first argument is the receiver)	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`, `int[]::new`	`() -> new ArrayList<>()`

Composition (§11.6): `f.andThen(g)` runs `f` then `g`; `f.compose(g)` runs `g` then `f` (with `plus1`, `times2` on 3: `8` vs `7`); `Predicate.and/or/negate`, `Predicate.not(String::isBlank)` (11); `Function.identity()`; `Comparator.comparing(...).thenComparing(...)`.

`Optional<T>` (§11.7) — a return type for "maybe no result":

Do	Don't
`opt.map(f).filter(p).orElse(default)`	`opt.get()` without checking (→ `NoSuchElementException`)
`orElseGet(() -> expensive())` for costly defaults	`orElse(expensive())` — the argument is **always** evaluated
`orElseThrow()` (10) when absence is a bug	`Optional` fields, parameters, collection elements
`ifPresentOrElse`, `or`, `stream()` (9)	returning `null` from a method that returns `Optional`
return an empty collection for "no results"	`Optional<List<…>>`
`Optional.ofNullable(x)` for maybe-null values	`Optional.of(null)` (→ `NullPointerException`)
`OptionalInt`/`OptionalDouble` for primitives	`Optional<Integer>` in hot numeric code

20.11 Streams and collectors

A **stream** is a one-shot, lazy pipeline over a source: **source → zero or more intermediate operations → one terminal operation** (§12.1, §12.2). Nothing runs until the terminal operation; elements flow one at a time, and short-circuiting operations stop early:

```
Stream.of(1, 2, 3, 4).peek(x -> trace.add("saw " + x)).filter(x -> x > 1).findFirst();
// result 2, trace [saw 1, saw 2] — elements 3 and 4 are never touched
```

Stage	Operations
Sources	`coll.stream()`, `Stream.of(…)`, `Arrays.stream(a)`, `IntStream.range(0, n)` / `rangeClosed`, `Stream.iterate(seed, hasNext, next)` (9), `Stream.generate(s)`, `Files.lines(p)`, `str.chars()`, `map.entrySet().stream()`
Intermediate, stateless	`filter`, `map`, `mapToInt/Long/Double`, `mapToObj`, `boxed`, `flatMap`, `mapMulti` (16), `peek` (debugging only)
Intermediate, stateful	`distinct`, `sorted`, `limit`, `skip`, `takeWhile` / `dropWhile` (9), `gather(…)` (24)
Terminal	`forEach`, `collect`, `toList()` (16), `toArray`, `reduce`, `count`, `min` / `max`, `sum` / `average` (primitive streams), `findFirst` / `findAny`, `anyMatch` / `allMatch` / `noneMatch`

Terminal facts (§12.4): on an empty stream `anyMatch` → `false`, `allMatch` and `noneMatch` → `true`; `reduce(identity, op)` returns `T`, `reduce(op)` returns `Optional<T>`; `IntStream.average()` returns `OptionalDouble`; `Stream.toList()` is unmodifiable but accepts `null` elements; reusing a consumed stream → `IllegalStateException`.

Collectors (§12.5) — the examples use five employees (eng: Ann 120, Bob 100; ops: Cid 90, Dee 95; hr: Eve 80):

Collector	Result in the checks	Watch out
`groupingBy(Emp::dept, TreeMap::new, counting())`	`{eng=2, hr=1, ops=2}`	default map is a `HashMap` (no order)
`partitioningBy(e -> e.salary() >= 100, mapping(Emp::name, toList()))`	`{false=[Cid, Dee, Eve], true=[Ann, Bob]}`	always has both keys, even for an empty stream
`toMap(Emp::dept, Emp::name)`	`IllegalStateException`	duplicate keys throw; `null` values → NPE
`toMap(Emp::dept, Emp::salary, Math::max, TreeMap::new)`	`{eng=120, hr=80, ops=95}`	merge function resolves duplicates
`joining(", ", "[", "]")`	`[Ann, Bob, Cid, Dee, Eve]`	elements must be `CharSequence`
`counting()`, `summingInt`, `averagingDouble`, `minBy`, `maxBy`, `summarizingInt`	numbers, `Optional`, `IntSummaryStatistics`	usually as downstream collectors
`mapping`, `filtering` (9), `flatMapping` (9), `collectingAndThen`, `teeing` (12)	adapted results	
`toList()`, `toSet()`, `toUnmodifiableList()` (10), `toCollection(TreeSet::new)`	collections	`Collectors.toList()` mutability is unspecified

Primitive streams (§12.6): `IntStream`, `LongStream`, `DoubleStream` have `sum`, `average`, `summaryStatistics`, `range`; convert with `mapToInt` / `boxed` / `mapToObj`. `IntStream.rangeClosed(1, 5).reduce(1, (a, b) -> a * b)` = `120`.

Gatherers (Java 24, §12.9): `stream.gather(g)` is a custom intermediate operation. Built-ins in `Gatherers`: `windowFixed(2)` on 1..5 → `[[1, 2], [3, 4], [5]]`; `windowSliding(n)`; `scan(() -> 0, Integer::sum)` on 1, 2, 3 → `[1, 3, 6]`; `fold`; `mapConcurrent(n, f)` (runs `f` on virtual threads, keeps order).

Rules and mistakes (§12.7, §12.8, §12.10, §12.11): lambdas in a pipeline should be stateless (not depend on anything that changes while the stream runs) and must not modify the source; don't use `peek`/`forEach` to fill an outside list — collect instead; `sorted`/`distinct` on infinite streams never finish; close streams from `Files.lines`/`list`/`walk`. Parallel streams pay off only for large, CPU-bound, easily split, stateless work (they use the common `ForkJoinPool`, the JVM's shared work-stealing thread pool); never for I/O. Loops remain better for early exits with complex state, checked exceptions and index arithmetic.

20.12 I/O and NIO.2

Which API? (§13.1):

Need	Use	Since
Name a file or directory	`Path.of("dir", "a.txt")` (was `Paths.get`)	7 (`Path.of` 11)
Small text file in one call	`Files.readString(p)`, `Files.writeString(p, s)`	11
Lines of a small file	`Files.readAllLines(p)`, `Files.write(p, lines)`	7
Huge text file	`try (Stream<String> s = Files.lines(p))` or `Files.newBufferedReader(p)`	8 / 7
Raw bytes from anywhere	`InputStream` / `OutputStream` (+ `transferTo`, `readAllBytes`)	1.0 (9)
Text from anywhere	`Reader` / `Writer` with an explicit charset	1.1
Big files, random access, memory mapping	`FileChannel` + `ByteBuffer`	1.4
Legacy code that wants `File`	`path.toFile()`, `file.toPath()`	7

`Path` (§13.2) never touches the disk: `Path.of("/a/b/../c/./d.txt").normalize()` → `/a/c/d.txt`; `Path.of("/a/b").relativize(Path.of("/a/c/d"))` → `../c/d`; `Path.of("/a").resolve("b.txt")` → `/a/b.txt`; `getFileName`, `getParent`, `getRoot`, `toAbsolutePath`, `startsWith`.

`Files` (§13.3, §13.4): `exists`, `isDirectory`, `createDirectories` (no error if present), `createFile` (fails if the file exists — `FileAlreadyExistsException`), `createTempFile`/`createTempDirectory`, `copy`/`move` (add `REPLACE_EXISTING`, `ATOMIC_MOVE`), `delete` (throws if missing or a non-empty directory) / `deleteIfExists`, `size`, `mismatch` (12), `list` / `walk` / `find` (lazy `Stream<Path>` — **close them**). `Files.size` counts **bytes**: `"üni\n"` is 6 bytes in UTF-8.

Streams and decorators (wrappers that add a feature, such as buffering, around another stream) (§13.5, §13.6): byte streams (`InputStream`/`OutputStream`) vs character streams (`Reader`/`Writer`); bridge with `InputStreamReader`/`OutputStreamWriter` and a charset; wrap in `BufferedReader`/`BufferedWriter`/`BufferedInputStream` (8 KB buffer by default) — unbuffered byte-at-a-time I/O is very slow, and a writer you forget to `flush`/`close` loses data. Always use try-with-resources.

Charsets (§13.7): the default charset is **UTF-8** since Java 18 (`Charset.defaultCharset()`); still pass `StandardCharsets.UTF_8` explicitly for files other systems produce. Mojibake = bytes decoded with the wrong charset.

Binary data (§13.9): `DataInputStream`/`DataOutputStream` for primitives; `FileChannel`, `ByteBuffer` (`flip` before reading what you wrote), memory-mapped files; `WatchService` for directory change events.

Serialization (§13.10): `implements Serializable`, `transient` fields are skipped, declare `serialVersionUID`. Deserialising untrusted data is a **security risk** (gadget chains: existing classes an attacker strings together to run code) — prefer JSON (a common text format for data) or protobuf (Protocol Buffers, Google's compact binary format); if you must, use deserialization filters (`ObjectInputFilter`, 9; context-specific filters, 17).

Also (§13.8, §13.11–§13.13): `Properties` for `key=value` config (`load`, `getProperty(k, default)`); JSON needs a library (Jackson, Gson) — the JDK has none; `System.in`/`out`/`err`, `Console`, `Scanner`, `IO` (25).

20.13 Concurrency

Threads (§14.1–§14.4):

```
Thread t = Thread.ofPlatform().name("worker").start(task);      // or new Thread(task).start()
Thread v = Thread.ofVirtual().start(task);                       // Java 21
t.join();                                                        // wait for it to finish
```

Call `start()`, not `run()` (which runs on the *current* thread). States: `NEW` → `RUNNABLE` → (`BLOCKED` on a monitor — the lock built into every object · `WAITING` on `wait`/`join`/`park` · `TIMED_WAITING` on `sleep`/timeouts) → `TERMINATED`. Interruption is cooperative: `t.interrupt()` sets a flag; blocking methods throw `InterruptedException` — rethrow it or restore the flag with `Thread.currentThread().interrupt()`, never swallow it. There is no safe way to kill a thread: `Thread.stop()` threw `UnsupportedOperationException` from Java 20 and was removed in JDK 26. The JVM exits when only daemon threads (background threads) remain.

The primitives (§14.5–§14.13):

Tool	Gives you	Use for	Remember
`synchronized` (method or block)	mutual exclusion **and** visibility; reentrant (the holder may lock again)	protecting shared mutable state, multi-variable invariants	instance method locks `this`, static method locks the `Class` object; released automatically
`volatile` field	visibility and ordering only	a stop flag, safely publishing a reference	`count++` on a volatile is **still a race**
`AtomicInteger`, `AtomicLong`, `AtomicReference`	lock-free single-variable updates via CAS (compare-and-set)	counters, sequence numbers	`compareAndSet(expected, new)` → `true` only if the value matched; `LongAdder` for very hot counters
`ReentrantLock`	`tryLock`, timeouts, interruptible, fair mode, several `Condition`s	when `synchronized` isn't flexible enough	**`unlock()` in `finally`**
`ReadWriteLock` / `StampedLock`	many readers or one writer / optimistic reads	read-mostly data	`StampedLock` is not reentrant
`wait` / `notifyAll`	guarded blocks	legacy code — prefer `java.util.concurrent`	call in a `while` loop, holding the lock
`ConcurrentHashMap`	thread-safe map with atomic `merge`, `compute`, `computeIfAbsent`, `putIfAbsent`	shared caches, counters	no `null` keys/values; iterators weakly consistent
`CopyOnWriteArrayList`	snapshot iteration, never throws CME	listeners, read-mostly lists	every write copies the array
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)	`put` waits when full, `take` waits when empty	producer–consumer (one thread makes items, another uses them)	prefer bounded queues
`CountDownLatch(n)`	wait until n `countDown()` calls	start/finish gates	one-shot
`CyclicBarrier(n)`	n threads wait for each other, repeatedly	phased algorithms	reusable
`Semaphore(k)`	at most k holders at a time	limit concurrent DB/API calls (also with virtual threads)	`release` in `finally`
`ThreadLocal` / `ScopedValue` (25)	per-thread value / immutable per-call context	request context	remove `ThreadLocal`s on pooled threads

Executors (§14.11): `try (ExecutorService pool = Executors.newFixedThreadPool(n)) { Future<T> f = pool.submit(callable); }` — `ExecutorService` is `AutoCloseable` since Java 19, and `close()` waits for submitted tasks. Factories: `newFixedThreadPool(n)` (unbounded queue!), `newCachedThreadPool()` (unbounded threads!), `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newWorkStealingPool()`, `newVirtualThreadPerTaskExecutor()` (21). For full control use `new ThreadPoolExecutor(core, max, keepAlive, unit, boundedQueue, factory, rejectionPolicy)`. Size CPU-bound pools (work limited by computation, not waiting) ≈ number of cores.

`CompletableFuture` (§15.1, §15.2):

Method	Does	Checked result
`supplyAsync(supplier[, executor])` / `runAsync`	start async work (default: common `ForkJoinPool`)	
`thenApply(fn)`	transform (like `map`)	
`thenCompose(fn)`	chain another async step (like `flatMap`)	`5` → `6`
`thenCombine(other, fn)`	combine two independent results	price 20 × qty 3 = `60`
`allOf(…)` / `anyOf(…)`	wait for all / the first	
`exceptionally(fn)`	recover on failure	`-1`
`handle((v, ex) -> …)` / `whenComplete`	always runs; can change result / can't	
`orTimeout(d, unit)` / `completeOnTimeout(v, d, unit)` (9)	time limits	
`join()` / `get()`	block for the result	`join` wraps failures in `CompletionException` (unchecked); `get` in `ExecutionException` (checked)

Virtual threads (Java 21, §15.4, §15.5): cheap threads scheduled by the JVM onto a few carrier (platform) threads; blocking I/O unmounts them. Use one per task (`newVirtualThreadPerTaskExecutor()`) for **I/O-bound**, high-concurrency work; they don't speed up CPU-bound work. Don't pool them; cap scarce resources with a `Semaphore`; avoid big `ThreadLocal` caches. They are always daemon threads. Since **Java 24** (JEP 491) blocking inside `synchronized` no longer pins the carrier; native calls that call back into blocking Java code, and class initialisation, still do.

Also modern (§15.3, §15.6, §15.7): Fork/Join (`RecursiveTask`, work stealing — idle threads take queued tasks from busy ones — used by parallel streams); **scoped values** are final in Java 25 (`ScopedValue.where(USER, "alice").call(() -> USER.get())` → `"alice"`, and unbound again afterwards); **structured concurrency** (`StructuredTaskScope.open()` plus a `Joiner` policy) is still **preview** in 25 (JEP 505) and kept changing in 26 (JEP 525) and 27 (JEP 533: built-in joiners make `join()` throw `ExecutionException`, `awaitAll()` removed) — don't write production code against it yet.

Java Memory Model (§15.8): without a *happens-before* edge another thread may never see your write. Edges: program order in one thread; unlock → later lock of the same monitor; volatile write → later read of that field; `Thread.start()` → actions in the thread; actions in a thread → a successful `join()`; `java.util.concurrent` hand-offs (queue put → take, `countDown` → `await`, task submit → run, task → `Future.get`); transitivity. Safe publication (making a new object visible to other threads only once fully built): `final` fields, volatile, locks, concurrent collections, static initialisers.

Liveness (§15.9): **deadlock** (threads wait for each other's locks forever — fix with one global lock order, `tryLock` with timeout, fewer locks; diagnose with `jstack` / `jcmd Thread.print`), **livelock** (threads keep reacting to each other without progress — add randomised back-off), **starvation** (a thread never gets CPU or a lock — fair locks, bounded work).

Choosing a model (§15.10): blocking I/O at scale → virtual threads; fan-out/fan-in with composition → `CompletableFuture` (structured concurrency once final); CPU-heavy data work → parallel streams / Fork/Join / fixed pool of ≈ cores; periodic jobs → `ScheduledExecutorService`; shared counters/caches → atomics, `LongAdder`, `ConcurrentHashMap`. For design-level concurrency case studies see the LLD handbook.

20.14 The JVM: memory, GC, flags and tools

How code runs (§1.6, §1.7, §16.1, §16.2, §16.8): `javac` → bytecode in `.class` files → class loaders load classes lazily (bootstrap → platform → application, **parent-first delegation**) → verify, prepare, resolve, initialise (static initialisers run once) → the interpreter runs bytecode; hot methods are

compiled by the tiered JIT (**C1** fast, then **C2** heavily optimising: inlining (copying a called method into its caller), devirtualisation (calling the one known implementation directly), escape analysis, loop optimisations, deoptimisation when assumptions break). Hence "warm-up". Class loaders in the checks: `String.class.getClassLoader()` is `null` (bootstrap), `java.sql` classes use the `platform` loader, your classes the `app` loader.

Memory areas (§16.3, §1.8):

Area	Per thread?	Holds	Size flag	When it runs out
Heap (young: eden, where new objects start, + survivors; old)	shared	all objects and arrays	`-Xms`, `-Xmx`, `-XX:MaxRAMPercentage`	`OutOfMemoryError: Java heap space`
Java stack	per thread	frames: locals, operand stack	`-Xss`	`StackOverflowError`
Metaspace (native memory; replaced PermGen in 8)	shared	class metadata	`-XX:MaxMetaspaceSize` (unlimited by default)	`OutOfMemoryError: Metaspace`
Code cache	shared	JIT-compiled code	`-XX:ReservedCodeCacheSize`	JIT stops compiling (slow, not a crash)
Direct / native memory	shared	direct `ByteBuffer`s, thread stacks, GC structures	`-XX:MaxDirectMemorySize`	`OutOfMemoryError: Direct buffer memory` / `unable to create native thread`

Object size basics (§16.4): 12-byte header (8 with `-XX:+UseCompactObjectHeaders`, Java 25; the default from JDK 27), 4-byte compressed references, padded to 8 bytes — `new Object()` is 16 bytes; `record Point(int x, int y)` 24.

Garbage collectors (§16.5–§16.7) — an object is garbage when it isn't reachable from GC roots (thread stacks, static fields, JNI handles):

Collector	Flag	Pauses	Choose it for
G1 (default since 9 on machines with ≥ 2 CPUs and ≥ 1792 MB; in all environments from JDK 27, JEP 523)	`-XX:+UseG1GC`	goal `-XX:MaxGCPauseMillis` (200 ms default); typically a few–tens of ms	most services — the default for a reason
Parallel	`-XX:+UseParallelGC`	longer, grows with live data	batch jobs: maximum throughput
Serial	`-XX:+UseSerialGC`	stop-the-world (all app threads paused), single thread	tiny heaps, 1 CPU, CLI tools (before JDK 27 the JVM picked it on machines with < 2 CPUs or < 1792 MB)
ZGC (generational; generational-only since 24)	`-XX:+UseZGC`	typically < 1 ms, independent of heap size	latency-sensitive services, huge heaps
Shenandoah (generational mode product in 25)	`-XX:+UseShenandoahGC`	sub-ms to a few ms	low latency; in most OpenJDK builds, not Oracle's
Epsilon	`-XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC`	none — never collects	benchmarks, ultra-short jobs

Key flags (§16.7):

Flag	Meaning
`-Xms512m -Xmx2g`	initial / maximum heap (set them equal for servers to avoid resizing)
`-XX:MaxRAMPercentage=75`	heap as % of container memory (default 25)
`-Xss1m`	thread stack size (default platform-dependent: about 1 MB on 64-bit Linux, 2 MB on macOS/AArch64)
`-Xlog:gc*` (or `-Xlog:gc:file=gc.log`)	unified GC logging (9+)
`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps`	heap dump on OOM (`OutOfMemoryError`)
`-XX:+PrintFlagsFinal`	print every flag's final value
`-XX:StartFlightRecording=duration=60s,filename=app.jfr`	record with JDK Flight Recorder at start-up
`-XX:TieredStopAtLevel=1`	C1 only: faster start-up, lower peak speed
`-ea`	enable `assert` statements
`--enable-preview`	allow preview features (needs `--release N` at compile time)
`-XX:+UseCompactObjectHeaders`	8-byte object headers (25, opt-in; on by default from JDK 27, JEP 534)

Diagnostic tools (§16.10, §16.11):

Tool	Use
`jps -l`	list running JVMs and PIDs (process IDs)
`jcmd <pid> help`	the Swiss-army knife: `VM.flags`, `GC.heap_info`, `GC.class_histogram`, `Thread.print`, `GC.heap_dump file.hprof`, `JFR.start` / `JFR.dump`, `VM.native_memory`
`jstat -gcutil <pid> 1000`	heap occupancy and GC counts every second
`jstack <pid>`	thread dump — look for `BLOCKED` and "Found one Java-level deadlock"
JFR + `jfr view hot-methods app.jfr` / JDK Mission Control	low-overhead production profiling
VisualVM, Eclipse MAT, async-profiler	GUI monitoring, heap-dump analysis (dominator tree), flame graphs (charts of where CPU time goes)
JMH	micro-benchmarks that avoid warm-up and dead-code traps (§16.9)
`jdeps`, `jlink`	module dependencies / build a trimmed runtime (§16.12)

Symptom → first tool: high CPU → JFR hot methods; latency spikes → GC log pauses; hangs → two or three thread dumps; memory growth → `jstat` old-gen trend + class histogram twice, then a heap dump; RSS (resident set size — the memory the OS says the process uses) far above heap → native memory tracking. Classic leaks: unbounded static caches, unregistered listeners, `ThreadLocal`s on pooled threads, mutated `HashMap` keys, class-loader leaks.

Modules (JPMS) (§16.12): `module-info.java` with `requires`, `exports`, `opens` (for reflection — code inspecting classes at run time), `uses`/`provides` (services); the class path is the *unnamed module*; `import module java.base;` (25) imports every exported package of a module.

20.15 Modern Java by version: 17 to 25

Final features only — the version that **finalised** each one (§17.2; JEP numbers in parentheses). The book's baseline before 17: `var` (10), `HttpClient` and `java Hello.java` (11), switch expressions and helpful NPE messages (14; the messages on by default since 15), text blocks (15), records, `instanceof` patterns and `Stream.toList()` (16).

Release	GA (general availability)	Language and libraries	JVM and tools
17 LTS (long-term support)	Sep 2021	sealed classes (409), enhanced pseudo-random generators (356), always-strict floating point (306), context-specific deserialization filters (415)	strong encapsulation of JDK internals (403), macOS/AArch64 port (391)
18	Mar 2022	UTF-8 by default (400), `@snippet` in Javadoc (413)	simple web server `jwebserver` (408); finalization deprecated for removal (421)
19, 20	Sep 2022, Mar 2023	previews and incubators (experimental API modules) only	Linux/RISC-V port (422, in 19)
21 LTS	Sep 2023	record patterns (440), pattern matching for `switch` (441), **virtual threads** (444), sequenced collections (431), key encapsulation mechanism API (452)	generational ZGC (439), warnings for dynamically loaded agents (451)
22	Mar 2024	unnamed variables and patterns `_` (456), Foreign Function & Memory API (454), multi-file source launch (458)	region pinning for G1 (423)
23	Sep 2024	Markdown documentation comments `///` (467)	ZGC generational by default (474); `sun.misc.Unsafe` memory access deprecated for removal (471)
24	Mar 2025	**stream gatherers** (485), Class-File API (484), ML-KEM (496) and ML-DSA (497) post-quantum crypto	virtual threads without `synchronized` pinning (491), AOT class loading and linking (483), non-generational ZGC removed (490), Security Manager permanently disabled (486)
25 LTS	Sep 2025	compact source files and instance `main` (512), module import declarations (511), flexible constructor bodies (513), **scoped values** (506), key derivation function API (510)	compact object headers (519, opt-in), generational Shenandoah (521), AOT ergonomics (514) and method profiling (515), JFR cooperative sampling (518) and method timing (520)

Still not final in Java 25 (§17.11): structured concurrency (fifth preview, 505; sixth in 26, seventh in 27), primitive types in patterns (third preview, 507), stable values (preview, 502; renamed *lazy constants* in 26), PEM encodings (preview, 470), Vector API (tenth incubator, 508), JFR CPU-time profiling (experimental, 509). **Withdrawn**: string templates (previewed in 21 and 22, removed in 23). Try a preview with `javac --release 25 --enable-preview` and `java --enable-preview`. After 25 (§17.11): JDK 26 (Mar 2026) finalised HTTP/3 in `HttpClient` (517) and removed the Applet API (504) and `Thread.stop`; JDK 27 (Sep 2026) made G1 the default everywhere (523) and compact object headers the default (534). LTS releases: 8, 11, 17, 21, 25 (every two years); the next is expected to be 29 in September 2027 (§1.12).

Upgrading 17 → 25 (§17.12): raise the JDK first and `--release` second; upgrade build plugins and bytecode libraries (Lombok, Mockito, Byte Buddy, Spring); run `jdeps --jdk-internals` and `jdeprscan`; read the start-up warnings (agents, JNI, `Unsafe`); drop removed flags such as `-XX:-ZGenerational`.

20.16 Testing, build and tooling commands

JDK tools (§1.4–§1.11, §18.12, §18.13):

Command	Does
`java -version`, `javac -version`	check the installed JDK
`javac --release 25 -d out $(find src -name '*.java')`	compile for a target Java version
`java -cp out:lib/* com.example.Main`	run with a class path (`;` on Windows)
`jar --create --file app.jar --main-class com.example.Main -C out .`	build an executable JAR; run with `java -jar app.jar`
`javadoc -d docs …`, `javap -c`, `jshell`	docs, bytecode, REPL
`jlink`, `jpackage`	custom runtime image / native installer
`bash tools_run_java.sh java`	this book's runner: compiles each chapter's `code/` with `--release 25` and runs every `main`

Maven (§18.2; this book: Maven 3.9.14, Gradle 9.4.1) — `pom.xml`; coordinates `groupId:artifactId:version`; scopes `compile`, `test`, `provided`, `runtime`; "nearest wins" for version conflicts (pin with `dependencyManagement`); pin every plugin version too. Lifecycle: `validate → compile → test-compile → test → package → verify → install → deploy` — running a phase runs every phase before it.

Command	Does
`mvn clean verify`	full build with all checks (what CI — continuous integration, the server that builds every change — runs)
`mvn test`, `mvn -Dtest=CartTest test`	run all / one test class (`CartTest#method` for one method)
`mvn package -DskipTests`	build the JAR without running tests
`mvn dependency:tree`	print the resolved dependency tree
`./mvnw …`	the Maven Wrapper: pinned Maven version, committed to the repo

Gradle (§18.3) — `build.gradle(.kts)`; configurations `implementation`, `testImplementation`, `runtimeOnly`; incremental and cached tasks.

Command	Does
`./gradlew build`	compile, test, assemble
`./gradlew test --tests 'ch18.shop.CartTest'`	run matching tests
`./gradlew run`	run the `application` plugin's main class
`./gradlew dependencies --configuration testRuntimeClasspath`, `./gradlew tasks`	inspect

JUnit Jupiter (§18.4, §18.5, §18.8) — the same API in JUnit 5 and JUnit 6 (6.0, 2025: Java 17 baseline, one version number for Platform/Jupiter/Vintage, JSpecify annotations, Vintage deprecated); this book ran JUnit 6.1.3:

Annotation	Meaning
`@Test`	a test method (package-private is fine)
`@BeforeEach` / `@AfterEach`	run around every test (a new test-class instance per test by default)
`@BeforeAll` / `@AfterAll`	once per class (`static`)
`@DisplayName`, `@Disabled`, `@Tag`, `@Timeout`, `@Nested`	naming, skipping, filtering, time limit, grouping
`@ParameterizedTest` + `@ValueSource`, `@CsvSource`, `@MethodSource`, `@EnumSource`, `@NullAndEmptySource`	one test, many inputs
`@TempDir Path dir`	fresh temporary directory
`@ExtendWith(MockitoExtension.class)`	plug in an extension

Assertions: `assertEquals(expected, actual)` (expected first), `assertTrue`, `assertNull`, `assertThrows(X.class, () -> …)` (returns the exception), `assertAll(…)` (report every failure), `assertIterableEquals`, `assertTimeout`. Console launcher: `java -jar junit-platform-console-standalone-<version>.jar execute --class-path out:libs --scan-class-path`.

Mockito (§18.6): `mock(Repo.class)` or `@Mock`; stub with `when(repo.find("a")).thenReturn(Optional.of(x))` / `thenThrow`; `verify(repo).save(any())`, `verify(repo, never()).delete(…)`, `times(2)`; `ArgumentCaptor`; `@InjectMocks`; stub only what the test needs (strict stubs flag the rest). Fakes beat mocks for simple collaborators. Mockito 5 (this book: 5.24.0) mocks `final` classes through an agent — on JDK 21+ add `-javaagent:mockito-core.jar` to the test JVM (Surefire `argLine`, Gradle `jvmArgs`) instead of letting it self-attach.

AssertJ (§18.7): `assertThat(list).hasSize(3).containsExactly("a", "b", "c")`; `assertThat(s).startsWith("J").contains("av")`; `assertThatThrownBy(() -> …).isInstanceOf(IllegalArgumentException.class).hasMessageContaining("negative")`; `extracting(Order::id)`; `usingRecursiveComparison()`.

Test design (§18.9): Arrange–Act–Assert; one behaviour per test; name tests after behaviour; FIRST (fast, independent, repeatable, self-validating, timely); inject clocks and random seeds; test pyramid (many unit, some integration — Testcontainers — few end-to-end — whole-system tests). Testcontainers starts throwaway real services (such as a database) in Docker containers for tests.

Logging (§18.10): code against SLF4J (`private static final Logger log = LoggerFactory.getLogger(Foo.class); log.info("order {} placed", id);` — placeholders, not concatenation), with Logback or Log4j 2 as the backend; levels `TRACE < DEBUG < INFO < WARN < ERROR`; `java.util.logging` is built in (`FINE`, `INFO`, `WARNING`, `SEVERE`). Pass the exception as the last argument to keep the stack trace.

JDBC (§18.11): `try (Connection c = dataSource.getConnection(); PreparedStatement ps = c.prepareStatement("SELECT … WHERE id = ?")) { ps.setLong(1, id); try (ResultSet rs = ps.executeQuery()) { while (rs.next()) … } }` — always `PreparedStatement` (prevents SQL injection — attackers smuggling SQL commands inside input values), a connection pool (HikariCP; a set of open database connections reused across requests) in real apps, `setAutoCommit(false)` + `commit`/`rollback` for transactions (groups of changes that succeed or fail together). SQL itself: the SQL handbook.

20.17 Interview question index

All 42 questions are answered in Chapter 19 §19.29, each with links back to the teaching section; claims marked ✔ there are proven by a runnable program. The coding side — the Java DSA toolkit (§19.1–§19.7) and twenty patterns from two pointers to math and geometry (§19.8–§19.27) — is in the same chapter. Pattern finder:

Problem says…	Pattern	Section
sorted input, pair with target sum, in place	two pointers	§19.8
longest/shortest contiguous substring or subarray	sliding window	§19.9
seen before, count, group, pair in unsorted data	hashing	§19.10
range sums, subarrays summing to k (negatives allowed)	prefix sums	§19.11
brackets, next greater element	stack / monotonic stack	§19.12
k largest, merge k lists, running median, best among currently eligible	heap, two heaps	§19.13
sorted, first true, minimum capacity such that	binary search (on the answer)	§19.14
meetings, overlaps, rooms	intervals	§19.15
fewest steps, islands, weighted route	BFS / DFS / Dijkstra	§19.16
prerequisites, order with dependencies	topological sort	§19.17
connected groups as edges arrive	union-find	§19.18
number of ways, min cost, longest subsequence	dynamic programming	§19.19
all subsets, permutations, N-Queens	backtracking	§19.20
many prefix queries, autocomplete	trie	§19.21
k-th from end, middle, cycle start, intersection, palindrome list	linked lists, fast/slow pointers	§19.22
validate BST, lowest common ancestor, path sum, serialise	tree recursion (top-down / bottom-up)	§19.23
jump game, gas station, candies, local best is safe	greedy	§19.24
k-th element without full sort, sort colours, sort a list	quickselect, Dutch flag, merge sort	§19.25
appears twice except one, count 1-bits, subsets of ≤ 20	bit manipulation	§19.26
spiral matrix, reverse digits, points on a line, Josephus	math and geometry	§19.27

Strings, objects and the language → §19.29

Q	Topic	Q	Topic
1	why `String` is immutable	7	`final` vs `finally` vs `finalize`
2	the string pool; objects made by `new String("hi")`	8	making a class immutable
3	`String` vs `StringBuilder` vs `StringBuffer`	9	interface vs abstract class
4	the `equals` / `hashCode` contract	10	records and their limits
5	`==` vs `equals`, the `Integer` cache	11	sealed classes
6	pass-by-value	12	overloading vs overriding

Collections → §19.29: Q13 `HashMap` internals · Q14 mutating a key · Q15 `HashMap` vs `Hashtable` vs `synchronizedMap` vs `ConcurrentHashMap` · Q16 how `ConcurrentHashMap` is thread-safe · Q17 `ArrayList` vs `LinkedList` · Q18 map orderings and an LRU cache · Q19 `Comparable` vs `Comparator` · Q20 fail-fast vs weakly consistent iterators · Q21 `ArrayDeque` vs `Stack` · Q22 `List.of` vs `Arrays.asList` vs `unmodifiableList`.

Exceptions → §19.29: Q23 checked vs unchecked · Q24 how try-with-resources works · Q25 when `finally` doesn't run.

Generics, lambdas and streams → **§19.29**: Q26 type erasure and its limits · Q27 PECS · Q28 streams vs loops · Q29 misusing `Optional` · Q30 functional interfaces and effectively final.

Concurrency → **§19.29**: Q31 `volatile` vs `synchronized` · Q32 happens-before · Q33 deadlock · Q34 virtual threads · Q35 `Runnable`/`Callable`, `Future`/`CompletableFuture` · Q36 why executors.

The JVM → **§19.29**: Q37 memory areas · Q38 GC and the default collector · Q39 memory leaks · Q40 class loading, interpreter and JIT · Q41 `static` and static initialisers · Q42 modern Java features to use in interviews.

More interview material: each chapter's **In the interview** sections, the modern-Java round in §17.13, and exam-level traps in the OCP Java 21 study notes.

20.18 Where next

You now know the language, the core library, the JVM and the tools. What to learn next depends on the job you want:

Direction	What it is	First step
Spring Boot	the dominant framework for Java back-ends (Boot 4.0, Nov 2025, on Spring Framework 7; 4.1 in 2026; Java 17+): dependency injection, auto-configuration, starters for web, data, security, and Actuator for operations (§18.14)	generate a project at start.spring.io, build a small REST API (a web API of URLs answered over HTTP) with JDBC or JPA (Jakarta Persistence, which maps objects to database tables), test it with JUnit, Mockito and Testcontainers
Jakarta EE (formerly Java EE)	the standard enterprise specifications — Servlet, CDI, JPA, JAX-RS (Jakarta REST), Bean Validation — implemented by WildFly, Open Liberty, Payara	learn JPA and CDI; much of Spring uses the same specifications (e.g. JPA via Hibernate)
Quarkus, Micronaut, Helidon	cloud-native frameworks with fast start-up and small memory, strong GraalVM native-image support (compiling Java ahead of time into a stand-alone executable)	port your Spring REST API to compare
Kotlin	a JetBrains language on the JVM, fully interoperable with Java: null-safe types, data classes, coroutines; Google's preferred language for Android since 2019	rewrite one small program; call it from Java
Other JVM languages	Scala (functional + OO, Spark), Groovy (Gradle scripts, scripting), Clojure (Lisp)	only if a job needs them
Android	mobile apps with the Android SDK (Kotlin-first today, Java supported)	Android Developers' basic course
Data and messaging	JDBC/JPA, Kafka (a distributed message log) clients, Spark, Flink (big-data processing engines)	the SQL handbook and the DDIA notes
Design	SOLID (five class-design principles), patterns, low-level design case studies in Java	the LLD handbook
Certification	Oracle Certified Professional: Java SE 21 Developer (1Z0-830)	the OCP Java 21 study notes
Performance	JFR, JMH, GC tuning, async-profiler	revisit Chapter 16 with a real service

Keep up to date by reading the JEP list for each release on openjdk.org and the dev.java articles; a new feature release ships every March and September.

20.19 Glossary (A–Z)

Each term has a short definition and a link to the section that teaches it. Backticks and symbols are ignored when sorting; *italic* names point to another entry. 706 terms.

A

- **Abstract class** — A class declared `abstract` that can't be instantiated; it may hold state, constructors and abstract methods for subclasses to implement. §6.5
- **Abstract method** — A method with no body that a concrete subclass (or implementing class) must implement. §6.5
- **Access modifier** — `private`, package-private (none), `protected` or `public` — controls where a member can be used. §5.6
- **Accessor (record)** — The generated method named like a record component, e.g. `x()` for `record Point(int x, int y)`. §6.8
- **Actuator** — The Spring Boot module that exposes health, metrics and info endpoints for operations. §18.14
- **Adjacency list** — A graph stored as, for each node, the list of nodes it has an edge to. §19.16
- **Algebraic data type** — A type defined as "exactly one of these variants, each with these fields" — in Java, a sealed interface plus records. §3.7, §6.7
- **Algorithm** — A precise, step-by-step procedure that solves a problem; its cost is described with *Big-O notation*. Ch 19
- **Allocation rate** — How fast a program creates new objects (MB per second); a high rate means more frequent young collections. §16.10
- **Amortised O(1)** — Constant cost on average over many calls, even if an occasional call (such as an `ArrayList` resize) is O(n). §10.12
- **Anagram** — A word made by rearranging another word's letters (*tea*, *eat*); grouped by sorting each word's letters. §19.10
- **Annotation** — Metadata written `@Name` on code (`@Override`, `@FunctionalInterface`, `@Test`) that the compiler, tools or frameworks read. §6.3, §18.4
- **Anonymous class** — A class declared and instantiated in one expression, `new Interface() { … }`, with no name. §6.11
- **AOT cache** — Ahead-of-time class loading and linking data (Java 24–25) that the JVM reuses to start faster. §17.9, §17.10
- **API** — Application Programming Interface: the classes and methods a library, the JDK or a service offers for other code to call. §1.1
- **`ArithmeticException`** — Unchecked exception for integer division by zero or overflow detected by `Math.*Exact` methods. §2.10, §7.5
- **Arrange–Act–Assert** — The three-part shape of a clear test: set up the inputs, run the code under test, then check the result. §18.9
- **Array** — A fixed-length, indexed, zero-based container of one element type; an object with a `length` field. §4.6
- **`ArrayDeque`** — A resizable circular-array `Deque`; the recommended stack and queue; rejects `null`. §10.6
- **`ArrayIndexOutOfBoundsException`** — Thrown when an array index is negative or ≥ `length`. §4.6
- **`ArrayList`** — A growable-array `List` with O(1) indexed access and amortised O(1) append; the default list. §10.2
- **`Arrays.asList`** — Returns a fixed-size `List` view backed by an array: `set` writes through, `add`/`remove` throw. §4.8, §10.10
- **Arrow label** — `case X ->` in a `switch` (Java 14): runs only that case, with no fall-through. §3.3
- **ASCII** — The original 7-bit character set of 128 English letters, digits and symbols; the first 128 Unicode characters are the same. §7.1, §13.7
- **`assert`** — A statement that checks a condition only when the JVM runs with `-ea`; disabled by default. §19.1
- **AssertJ** — A fluent assertion library: `assertThat(x).isEqualTo(y)`. §18.7

- **Associativity** — The grouping order for operators of equal precedence — left for most, right for assignment and the ternary. §2.9
- **Asynchronous** — Started now and finished later without the caller waiting; the result arrives through a `Future`, a callback or a `CompletableFuture`. §15.1, §15.2
- **Atomic variable** — A class such as `AtomicInteger` whose single-variable updates are atomic without locks, using CAS. §14.10
- **Atomicity** — An operation is atomic when other threads can see it only as not started or fully done, never half-done. §14.5
- **Autoboxing** — Automatic conversion from a primitive to its wrapper (`int` → `Integer`); unboxing is the reverse. §7.6
- `AutoCloseable` — The interface with `close()` that try-with-resources calls automatically. §8.6

B

- **Backing array** — The array behind an `ArrayList` or an `Arrays.asList` view. §10.2, §10.10
- **Backtracking** — Searching every candidate solution by making a choice, exploring further, then undoing the choice to try the next one. §19.20
- **Banker's rounding** — `RoundingMode.HALF_EVEN`: a tie rounds to the even neighbour (2.345 → 2.34), which avoids drift in large sums. §7.7
- **Behaviour parameterisation** — Passing behaviour (today, a lambda) as an argument so one method can serve many cases. §11.1
- **Bellman-Ford** — A shortest-path algorithm that, unlike Dijkstra's, allows negative edge weights; slower at O(V · E). §19.16
- **Benchmark** — A measured timing of code; a fair Java microbenchmark needs warm-up and a harness such as *JMH*. §16.9
- **BFS (breadth-first search)** — Visiting a graph level by level from a start node using a queue; it finds shortest paths when every edge costs the same. §19.16
- `BigDecimal` — Immutable arbitrary-precision decimal number with an explicit scale; used for money. §7.7
- `BigInteger` — Immutable arbitrary-size integer. §7.7
- **Big-O notation** — How an operation's cost grows with input size n: O(1), O(log n), O(n), O(n log n), O(n²). §10.12, §19.7
- **Binary search** — Finding a key in sorted data by halving the range each step, O(log n). §4.8, §19.14
- **Binary search tree (BST)** — A binary tree where every key in a node's left subtree is smaller, and every key in its right subtree larger, than the node's key. §19.23
- **Binary tree** — A tree in which every node has at most two children, called left and right. §19.23
- **Bit manipulation** — Solving problems with bitwise operators on the binary form of integers — masks, shifts and xor tricks. §19.26
- **Bitmask** — An integer whose individual bits act as on/off flags, for example one bit per item of a subset. §19.26
- **Bitwise operators** — `&`, `\|`, `^`, `~`, `<<`, `>>`, `>>>` — operate on the individual bits of integers. §2.8
- `BLOCKED` — The thread state of a thread waiting to acquire a monitor held by another thread. §14.3
- `BlockingQueue` — A queue whose `put` waits when full and `take` waits when empty; the core of producer–consumer. §14.12
- **BOM** — (1) Byte order mark: optional bytes at the start of a text file that announce its encoding. (2) In Maven, a "bill of materials" POM that pins versions for a family of libraries. §13.7, §18.2
- **Bootstrap class loader** — The JVM's built-in loader for core modules such as `java.base`; shown as `null` by `getClassLoader()`. §16.2
- **Bounded type parameter** — A type parameter with a limit, e.g. `<T extends Number>`. §9.5
- **Boxing** — Wrapping a primitive value in its wrapper object. §7.6
- `break` — Leaves the innermost loop or `switch` (or a labelled statement with `break label`). §3.9
- **Bridge method** — A hidden method `javac` generates so that overriding still works after generic types are erased. §9.7

- **Brute force** — Solving a problem by trying every possibility; usually too slow, but a good first answer to improve on. §19.28
- **Bucket** — One slot of a hash table's array, holding the entries whose hashes map to that index. §10.5
- **Buffer** — A block of memory that collects data so it can be read or written in large chunks rather than byte by byte. §13.6, §13.9
- **Buffering** — Reading or writing data in blocks through an in-memory buffer to avoid many slow system calls. §13.6
- **Build tool** — Software (Maven, Gradle) that resolves dependencies, compiles, tests and packages a project. §18.2, §18.3
- **Byte Buddy** — A library for generating and changing classes at run time; Mockito uses it to build mocks. §17.12
- **Byte stream** — `InputStream` / `OutputStream`: I/O in raw 8-bit bytes. §13.5
- `ByteBuffer` — A NIO container of bytes with position, limit and capacity, used with channels. §13.9
- **Bytecode** — The platform-independent instruction set in `.class` files that the JVM executes. §1.6

C

- **C1 / C2** — HotSpot's two JIT compilers: C1 compiles quickly with light optimisation, C2 slowly with heavy optimisation. §1.7, §16.8
- **Cache** — A fast store of recently used or precomputed results kept so the next request is cheaper; it needs a size limit or eviction rule, or it becomes a memory leak. §10.4, §16.11, System Design Fundamentals
- **Call stack** — The stack of frames for the method calls a thread currently has in progress: a call pushes a frame, a return pops it. §4.5, §16.3
- `Callable` — A task interface `V call() throws Exception` that returns a value; see `Runnable`. §14.11
- **Callback** — A function handed to other code so that code can call it later, for example the lambda given to `thenApply`. §15.1, §15.2
- **Canonical constructor** — A record's constructor that takes every component in declaration order; generated unless you write it. §6.8
- **Card table / remembered set** — GC bookkeeping that records which old-generation areas point into the young generation, so a young collection needn't scan the whole heap. §16.5
- **Carrier thread** — A platform thread on which the JVM mounts virtual threads to run them. §15.4
- **CAS (compare-and-set)** — An atomic CPU instruction that updates a value only if it still equals an expected value. §14.10
- **Casting** — Explicit conversion `(Type) value` — narrowing a primitive, or checking an object's run-time type. §2.7, §6.4
- **CDI** — Jakarta Contexts and Dependency Injection: the Jakarta EE standard for dependency injection. §1.2
- **CDS (Class Data Sharing)** — Saving already-parsed classes in an archive that later JVM runs map in, so they start faster; the *AOT cache* builds on it. §16.2
- **Channel** — An NIO connection to a file or socket (such as `FileChannel`) that reads and writes through `ByteBuffer`s. §13.9
- `char` — The 16-bit unsigned primitive holding one UTF-16 code unit (0 … 65 535). §2.3
- **Character stream** — `Reader` / `Writer`: I/O in 16-bit characters, decoded with a charset. §13.5
- **Charset** — A mapping between characters and bytes, such as UTF-8; the JDK default since Java 18 is UTF-8. §13.7
- **Checked exception** — An `Exception` outside the `RuntimeException` branch; must be caught or declared with `throws`. §8.2, §8.3
- **Check-then-act** — Testing a condition and then acting on it as two separate steps; another thread can change things in between, so the pair must be made atomic. §14.5
- **CI (continuous integration)** — A server that automatically builds and tests every change pushed to a shared repository. §18.1, §18.2
- **Circular array** — An array used as a ring whose indexes wrap from the end back to the start, so a queue can grow at both ends cheaply (`ArrayDeque`). §10.6

- **Class** — A blueprint that defines fields, constructors and methods; objects are its instances. §5.1
- **Class file** — The `.class` file `javac` produces, containing bytecode and a constant pool. §1.6
- **Class initialisation** — Running a class's static field initialisers and `static {}` blocks, once, on first active use. §5.10, §16.2
- **Class loader** — The object that finds and loads class bytes; loaders delegate to their parent first. §16.2
- **Class path** — The list of directories and JARs where the JVM looks for classes (`-cp`). §1.5
- **Class-File API** — The `java.lang.classfile` API (final in Java 24) for reading and writing class files. §17.9
- `Clock` — `java.time.Clock`: a replaceable source of "now"; inject it so tests can fix the time. §7.8, §18.9
- **Closure** — A function together with the variables it captured from its surrounding scope; lambdas capture effectively final locals. §11.5
- **CME** — Short for `ConcurrentModificationException`. §10.7
- **Code cache** — Memory where the JIT stores compiled machine code. §16.3
- **Code coverage** — The share of lines or branches that the tests execute, measured with tools such as JaCoCo; high coverage doesn't prove good tests. §18.9
- **Code point** — The number Unicode gives one character; a code point above U+FFFF needs two `char`s (a *surrogate pair*) in Java. §7.2
- **Code unit** — One 16-bit `char` of UTF-16 text; most characters take one code unit, some take two. §2.3, §7.2
- **Coffman conditions** — The four conditions that must all hold for a deadlock: mutual exclusion, hold and wait, no preemption, circular wait. Break any one to prevent it. §15.9
- **Collection** — The root interface of lists, sets and queues (maps are separate). §10.1
- `Collections` — Utility class with `sort`, `reverse`, `shuffle`, `frequency`, `unmodifiableList` and more. §10.9
- **Collector** — A recipe for accumulating stream elements into a result, e.g. `groupingBy`, `toMap`, `joining`. §12.5
- **Coloured pointers** — ZGC's trick of storing a few GC status bits inside object references themselves. §16.5
- **Compact constructor** — A record constructor without a parameter list, used to validate or normalise components. §6.8
- **Compact object headers** — Java 25 option (`-XX:+UseCompactObjectHeaders`) that shrinks object headers from 12 to 8 bytes; on by default from JDK 27. §16.4, §17.10
- **Compact source file** — A Java 25 source file with top-level methods and no class declaration; run with `java File.java`. §4.9, §17.10
- **Compaction** — Moving live objects next to each other so free memory forms one large block. §16.5
- `Comparable` — Interface whose `compareTo` defines a class's natural order. §10.8
- `Comparator` — A separate object whose `compare` defines an ordering; built with `comparing`, `thenComparing`, `reversed`. §10.8
- **Compiler** — A program that translates source code into another form: `javac` turns Java into bytecode, the JIT turns bytecode into machine code. §1.5, §1.7
- **Compile-time constant** — A `final` primitive or `String` initialised with a constant expression; folded by `javac`. §7.1
- `CompletableFuture` — A `Future` you can complete manually and compose asynchronously (`thenApply`, `thenCompose`, `allOf`). §15.2
- `CompletionException` — The unchecked wrapper `CompletableFuture.join()` throws when the computation failed. §15.2
- **Composition** — Building a class from other objects held in fields and delegating work to them — often preferred to inheritance. §6.1
- **Compound assignment** — Operators such as `+=`, which include an implicit cast back to the variable's type. §2.8

- **Compressed oops** — Storing object references in 32 bits on a 64-bit JVM (heaps up to about 32 GB) to save memory. §16.4
- **Concurrency** — Several tasks in progress over the same period, possibly interleaved on one core. §14.1
- **Concurrent collection** — A collection from `java.util.concurrent`, such as `ConcurrentHashMap` or `BlockingQueue`, designed for safe use by many threads at once. §14.12
- `ConcurrentHashMap` — A thread-safe, highly scalable hash map with atomic `merge`/`compute`; no `null` keys or values. §14.12
- `ConcurrentModificationException` — Thrown by fail-fast iterators when a collection is structurally changed during iteration. §10.7
- `Condition` — A waiting queue tied to an explicit `Lock` (`await`, `signal`); one lock can have several conditions. §14.9
- **Connected component** — A group of graph nodes that can all reach each other; counted with DFS, BFS or union-find. §19.16, §19.18
- **Connection pool** — A set of open database connections reused across requests, because opening one is slow (e.g. HikariCP). §18.11
- **Constant folding** — The compiler working out a constant expression such as `"ja" + "va"` at compile time. §7.1
- **Constant pool** — The table inside a class file that holds the literals and symbolic names its bytecode refers to. §1.6
- **Constructor** — A special method with the class's name that initialises a new object. §5.3
- **Constructor chaining** — Calling another constructor with `this(...)` or the parent's with `super(...)`. §5.3, §6.1
- **Container** — A lightweight, isolated package of an application with its own CPU and memory limits (e.g. Docker); the JVM reads those limits to size itself. §16.3, §16.7
- **Context switch** — The OS pausing one thread and running another on the same CPU core. §14.1
- **Contiguous** — Stored or taken next to each other with no gaps, like the elements of an array or a subarray. §4.6, §19.9
- `continue` — Skips to the next iteration of the innermost (or labelled) loop. §3.9
- `CopyOnWriteArrayList` — A thread-safe list that copies its array on every write; iterators see a snapshot. §14.12
- `CountDownLatch` — A one-shot synchronizer: threads `await()` until `countDown()` has been called n times. §14.13
- **Covariant return type** — An overriding method may return a subtype of the overridden method's return type. §6.3
- **CPU** — Central processing unit: the processor that executes instructions; its cores each run one thread at a time. §14.1
- **CPU-bound** — Work limited by computation rather than by waiting for I/O; more threads than cores won't speed it up. §12.8, §15.4
- **Critical section** — Code that touches shared mutable state and must not run in two threads at once. §14.1
- **Currying** — Turning a function of several arguments into a chain of one-argument functions. §11.8
- `CyclicBarrier` — A reusable synchronizer where n threads wait for each other before all continue. §14.13

D

- **Daemon thread** — A background thread that doesn't keep the JVM alive; virtual threads are always daemon. §14.2
- **DAG (directed acyclic graph)** — A graph whose edges have a direction and that contains no cycle; it always has a topological order. §19.17
- **Dangling else** — The question of which `if` an `else` belongs to when braces are missing; Java pairs it with the nearest `if`. §3.1
- **Data race** — Two threads access the same variable without a happens-before order and at least one writes. §15.8

- **Data structure** — A way of organising data in memory — array, list, hash table, tree, heap — chosen for the operations you need to be fast. §10.1, Ch 19
- `DateTimeFormatter` — Immutable, thread-safe formatter and parser for `java.time` values. §7.11
- **Daylight saving time (DST)** — The seasonal shift of clocks in many regions, which makes some local times skip or repeat. §7.10
- **Dead-code elimination** — The JIT removing computation whose result is never used — the classic trap in hand-written benchmarks. §16.9
- **Deadlock** — Threads wait forever for locks each other holds. §15.9
- **Declarative style** — Describing what result you want (filter, map, group) instead of writing the loop that produces it. §11.8, §12.11
- **Decorator** — A wrapper that adds behaviour to an object while offering the same interface, e.g. `BufferedReader` around a `Reader`. §13.5
- **Default constructor** — The no-argument constructor the compiler adds when a class declares no constructor. §5.3
- **Default method** — An interface method with a body (Java 8), inherited by implementing classes unless overridden. §6.6
- **Default value** — The initial value of a field or array element: `0`, `false` or `null`; local variables have none. §2.2, §2.3
- **Defensive copy** — Copying a mutable input or output so outside code can't change an object's internal state. §5.9
- **Definite assignment** — The compiler rule that a local variable must certainly have been assigned before it is read. §2.2
- **Deoptimisation** — The JIT discarding compiled code whose speculative assumptions stopped holding. §16.8
- **Dependency** — A library your project uses; build tools download it together with the libraries it needs in turn. §18.1, §18.2
- **Dependency injection** — Supplying an object's collaborators from outside (usually through the constructor) instead of creating them inside. §18.6, §18.14
- `Deque` — A double-ended queue: insert and remove at both ends; used as stack or queue. §10.6
- **Deserialisation** — Rebuilding objects from bytes; dangerous with untrusted input. §13.10
- **Devirtualisation** — The JIT replacing an overridable (virtual) call with a direct call when only one implementation has been seen. §16.8
- **DFS (depth-first search)** — Visiting a graph by following each path as deep as possible before backing up, using recursion or an explicit stack. §19.16
- **Diamond operator** — `<>` in `new HashMap<>()` — lets the compiler infer the type arguments. §9.2
- **Difference array** — An array of "change starts here" marks: add v at l and −v at r + 1, then one running sum applies every range update. §19.11
- **Dijkstra's algorithm** — Finds the cheapest paths from one node in a graph with non-negative edge weights, using a priority queue. §19.16
- **Direct memory** — Native memory outside the heap used by direct `ByteBuffer`s. §16.3
- **Divide and conquer** — Splitting a problem into smaller parts, solving each (often recursively or in parallel) and combining the results. §4.5, §15.3
- **Docker** — The most widely used tool for building and running containers. §16.3, §18.9
- `double` — The 64-bit IEEE 754 floating-point primitive; the default for decimals. §2.3, §2.11
- `do-while` — A loop that runs its body once before testing the condition. §3.8
- **Downcasting / upcasting** — Upcasting treats an object as one of its supertypes (automatic); downcasting casts back to a subtype and can throw `ClassCastException`. §6.4
- **Downstream collector** — A collector applied to each group inside `groupingBy` or `partitioningBy`, such as `counting()`. §12.5
- **DSA** — Data structures and algorithms — the subject of coding interviews. Ch 19
- **DSL** — Domain-specific language: a small language for one job, such as Gradle's Kotlin build scripts. §18.3

- **DTO** — Data transfer object: a plain data carrier passed between layers or over the network; a record is a natural fit. §6.8
- **Dummy (sentinel) node** — A placeholder node in front of a linked list's head, so changing the head needs no special case. §19.22
- `Duration` — A `java.time` amount of time in seconds and nanoseconds (e.g. `PT1H30M`). §7.10
- **Dutch national flag** — Splitting an array into three regions (less than, equal to, greater than a value) in one pass. §19.25
- **Dynamic dispatch** — Choosing which overridden method to run from the object's run-time class. §6.4
- **Dynamic programming (DP)** — Solving a problem from the answers to overlapping smaller subproblems, computing each answer once (memoisation or tabulation). §19.19

E

- **Eclipse Temurin** — The Adoptium project's free, widely used OpenJDK build. §1.2
- **Eden** — The part of the young generation where new objects are first allocated. §16.5
- **Effectively final** — A local variable that is never reassigned after initialisation; lambdas may capture only these. §11.5
- **Encapsulation** — Hiding an object's state behind methods so its invariants can't be broken from outside. §5.7
- **Encoding** — The rules that turn characters into bytes and back; see *charset*. §13.7
- **Encounter order** — The order in which a stream meets its elements: a `List` has one, a `HashSet` does not. §12.4
- **Endianness** — The byte order of multi-byte numbers: big-endian (Java's default) stores the most significant byte first. §13.9
- **End-to-end test** — A test of the whole running system through its real entry points, as a user would use it. §18.9
- **Enhanced `for`** — The for-each loop, `for (T x : iterable)`. §3.8
- **Enum** — A type with a fixed set of named instances; may have fields, constructors and methods. §6.9
- `EnumMap` / `EnumSet` — Very compact, fast map/set implementations for enum keys/elements. §10.4, §10.3
- **Environment variable** — A named setting the operating system passes to every program it starts, such as `JAVA_HOME` or `PATH`. §1.4
- **Epoch** — The fixed instant 1970-01-01T00:00:00Z from which `Instant` and `System.currentTimeMillis()` count. §7.10
- **Epsilon GC** — A no-op collector that never frees memory; for testing. §16.5
- `equals` — The method that defines logical equality; must be consistent with `hashCode`. §6.13
- **Equivalence class** — A group of inputs that the code treats the same way, so one test (plus the boundaries) covers it. §18.9
- **Erasure** — See *type erasure*. §9.7
- `Error` — The `Throwable` branch for serious JVM problems such as `OutOfMemoryError`; unchecked, normally not caught. §8.2
- **Escape analysis** — A JIT analysis proving an object doesn't leave a method, enabling scalar replacement and lock elision. §16.8
- **Escape sequence** — A backslash code for a special character in a literal: `\n`, `\t`, `\"`, `\\`, `A`. §2.4
- **Exception** — An object signalling an abnormal condition, thrown with `throw` and caught with `catch`. §8.1
- **Exception chaining** — Wrapping a low-level exception as the cause of a higher-level one. §8.8
- **Exception translation** — Catching an exception at a layer boundary and throwing one that fits the higher layer's abstraction. §8.8
- **Executable JAR** — A JAR whose manifest names a `Main-Class`, so it runs with `java -jar app.jar`. §18.12
- `ExecutorService` — A thread-pool abstraction that accepts tasks and returns `Future`s; `AutoCloseable` since Java 19. §14.11

- **Exhaustive switch** — A `switch` whose cases cover every possible value (all enum constants, all permitted subtypes, or `default`). §3.3, §3.6
- **Explicit lock** — A `java.util.concurrent.locks.Lock` such as `ReentrantLock`, acquired and released in code. §14.9
- **Expression** — Code that produces a value, such as `a + b`, `x > 0` or a method call; a statement is a complete instruction. §2.1

F

- **Factory method** — A static method that creates instances (`List.of`, `Integer.valueOf`); unlike a constructor it has a name and may cache or return a subtype. §5.3
- **Fail-fast iterator** — An iterator that throws `ConcurrentModificationException` on a best-effort basis when its collection changes structurally. §10.7
- **Failure atomicity** — A method that fails leaves its object exactly as it was before the call. §8.7, §8.10
- **Fair lock** — A lock that grants access in waiting order, avoiding starvation at some cost in speed. §14.9
- **Fake** — A working but simplified test double, such as an in-memory repository. §18.6
- **Fall-through** — In a classic `case X:` switch, execution running on into the next case when there is no `break`. §3.2
- **Fast and slow pointers** — Two references moving through a list at different speeds, used to find the middle or detect a cycle. §19.22
- **Fat JAR** — A single JAR containing the application and all of its dependencies (also "uber JAR"). §18.12
- **Feature release** — One of the two Java versions released every year, in March and September. §1.12
- **Field** — A variable declared in a class that holds an object's state (or, if `static`, the class's). §5.1
- **FIFO / LIFO** — First in, first out (a queue) / last in, first out (a stack). §10.1, §10.6
- **File descriptor** — The operating system's small-integer handle for an open file or socket; each process may hold only a limited number. §16.13
- **`final`** — Modifier: a final variable is assigned once, a final method can't be overridden, a final class can't be extended. §5.8
- **Finalization** — The deprecated-for-removal `finalize()` mechanism (Java 18, JEP 421); use try-with-resources or `Cleaner`. §17.12
- **`finally`** — A block after `try` that always runs when control leaves the `try`. §8.4
- **Flaky test** — A test that sometimes passes and sometimes fails with no code change, usually because of timing, randomness or shared state. §18.9
- **Flexible constructor bodies** — Java 25 feature allowing statements that don't use `this` before `super(...)` or `this(...)`. §17.10
- **`float`** — The 32-bit IEEE 754 floating-point primitive (~7 significant digits). §2.3
- **Floating-point error** — Rounding error because binary fractions can't represent most decimals exactly (`0.1 + 0.2`). §2.11
- **Floyd's cycle-finding algorithm** — Detecting a loop with a slow pointer (1 step) and a fast pointer (2 steps): they meet only if there is a cycle. §19.22
- **Foreign Function & Memory API** — The `java.lang.foreign` API (final in Java 22) to call native code and manage off-heap memory, replacing JNI. §17.7
- **Fork/Join** — A framework for recursive divide-and-conquer tasks run by a work-stealing `ForkJoinPool`. §15.3
- **Formatter specifier** — A pattern such as `%5.2f` or `%-10s` in `String.format`/`printf`. §7.4
- **Fragile base class** — A subclass breaking because its superclass's internals changed — a risk of inheritance across teams or libraries. §6.1
- **Frame (stack frame)** — The record one method call keeps on its thread's stack: parameters, local variables and where to return. §4.5, §16.3
- **`fsync`** — An operating-system call that forces written data out of memory caches onto the disk. §16.13

- **Full GC** — A collection of the whole heap, usually with a long stop-the-world pause. §16.5
- **Functional interface** — An interface with exactly one abstract method; a lambda or method reference can implement it. §11.2
- **Functional programming** — A style built from pure functions, immutable data and functions passed around as values. §11.8
- `Future` — A handle to an asynchronous result, obtained with a blocking `get()`. §15.1

G

- **GA (general availability)** — The official release of a Java version for production use. §20.15
- **Garbage collection (GC)** — Automatic reclamation of memory used by objects that are no longer reachable. §1.8, §16.5
- **Garbage-First (G1)** — The default collector since Java 9 (in all environments from JDK 27); region-based, with a pause-time goal. §16.5
- **Gatherer** — A custom intermediate stream operation used with `stream.gather(...)` (final in Java 24). §12.9
- **GC roots** — Starting points for reachability: thread stacks, static fields, JNI references. §16.5
- **GCD / LCM** — Greatest common divisor (the largest number dividing both) / least common multiple (the smallest number both divide). §19.27
- **Generational hypothesis** — Most objects die young, so collectors split the heap into young and old generations. §16.5
- **Generic method** — A method that declares its own type parameters: `static <T> T first(List<T> xs)`. §9.4
- **Generics** — Type parameters on classes, interfaces and methods for compile-time type safety without casts. §9.1
- **Getter / setter** — Methods that read (`getName()`) or change (`setName(...)`) a private field. §5.7
- **GIL** — Python's global interpreter lock, which lets one thread run Python code at a time; Java has none, so its threads truly run in parallel. §14.14
- **GraalVM** — Oracle's JDK distribution with an advanced JIT and *Native Image*. §1.2, §18.14
- **Gradle** — A build tool using a Groovy or Kotlin DSL with incremental, cached tasks. §18.3
- **Graph** — A set of nodes joined by edges, which may have a direction and a weight (cost); trees, grids and road maps are graphs. §19.16
- **Greedy algorithm** — Building an answer by always taking the choice that looks best right now and never undoing it; correct only when the greedy-choice property holds. §19.24
- **Greedy-choice property** — Some optimal answer starts with the locally best choice — the condition that makes a greedy algorithm correct. §19.24
- **Guard clause** — An early `return` (or `throw`) at the top of a method for special cases, keeping the main logic unindented. §3.1
- **Guard (pattern)** — A `when` condition attached to a `case` pattern. §3.6
- **GUI** — Graphical user interface: windows, buttons and menus. §16.10

H

- **Handle-or-declare rule** — A checked exception must be caught or listed in the method's `throws` clause. §8.3
- **Happens-before** — The Java Memory Model relation guaranteeing one action's effects are visible to another. §15.8
- **Hash table** — An array of buckets indexed by each key's hash code, giving O(1) average lookups; `HashMap` is one. §10.5
- `hashCode` — Returns an `int` used by hash-based collections; equal objects must return equal hash codes. §6.13
- `HashMap` — An unordered hash-table map with average O(1) operations; allows one `null` key. §10.4, §10.5
- `HashSet` — A set backed by a `HashMap`; unordered, average O(1). §10.3
- **Heap** — The JVM memory area where all objects and arrays live, shared by all threads and garbage-collected. §16.3

- **Heap (data structure)** — A tree kept in an array where each parent is ≤ its children (a min-heap), so the smallest item is always at the root; O(log n) insert and remove. Unrelated to the JVM *heap*. §10.6, §19.13
- **Heap dump** — A snapshot of every object in the heap, analysed with Eclipse MAT or VisualVM. §16.10, §16.11
- **Heap pollution** — A variable of a parameterized type referring to an object of the wrong type, caused by unchecked operations. §9.8
- **Helpful NullPointerException** — NPE messages that name exactly which expression was `null` (Java 14, on by default since 15). §8.1
- **Hexadecimal** — Base-16 notation with digits 0–9 and a–f, written `0x2A` in Java. §2.4
- **Hibernate** — The most popular JPA implementation, which maps Java objects to database tables. §18.11, §18.14
- **Hiding** — A static method or field in a subclass with the same name as the parent's; chosen by compile-time type, not overridden. §6.2
- **Higher-order function** — A function that takes or returns another function, such as `map` or `Comparator.comparing`. §11.8
- **HikariCP** — A fast JDBC connection pool; Spring Boot's default. §18.11
- **HotSpot** — The JVM inside OpenJDK and most Java distributions, named after its compilation of "hot" code. §1.3
- **HTTP** — Hypertext Transfer Protocol: the request/response protocol of the web, spoken by browsers, REST APIs and Java's `HttpClient`. §18.14
- **Humongous object** — In G1, an object at least half a region in size, which gets regions of its own. §16.5

I

- **IDE** — Integrated development environment, such as IntelliJ IDEA, Eclipse or VS Code: editor, compiler, debugger and tools in one. §1.11
- **Identity** — Whether two references point to the very same object (`==`), as opposed to being equal in value (`equals`). §6.13
- **`IdentityHashMap` / `WeakHashMap`** — Special maps: `IdentityHashMap` compares keys with `==`; `WeakHashMap` drops an entry once nothing else references its key. §10.4
- **IEEE 754** — The international standard for binary floating-point numbers that `float` and `double` follow. §2.11
- **`IllegalArgumentException`** — Unchecked exception for a method argument that violates a precondition. §8.7
- **`IllegalStateException`** — Unchecked exception for a call made when the object is in the wrong state. §8.7
- **Immutable object** — An object whose state can't change after construction, such as `String` or a record of immutable values. §5.9
- **Incidental indentation** — The common leading whitespace that a text block strips from every line. §2.12
- **Incubator module** — An experimental API in a `jdk.incubator.*` module, e.g. the Vector API. §17.1
- **In-degree** — The number of edges pointing into a node; Kahn's algorithm starts from the nodes with in-degree 0. §19.17
- **Inheritance** — A subclass acquiring a superclass's members with `extends`; Java has single class inheritance. §6.1
- **Initialisation order** — Static initialisers once per class, then for each object: parent constructor, field initialisers and instance blocks, constructor body. §5.10
- **Inlining** — The JIT copying a callee's body into its caller to remove call overhead and enable further optimisation. §16.8
- **Inner class** — A non-static nested class; each instance holds a reference to an enclosing instance. §6.10
- **In-place** — Changing the input directly with O(1) extra memory instead of building a new copy. §19.8

- **Instance** — An object created from a class; instance members belong to each object, `static` ones to the class. §5.1
- **Instance initialiser block** — A `{ … }` block in a class body that runs for every new object, before the rest of the constructor body. §5.10
- **Instance main method** — A Java 25 `main` that is not `static` and may take no parameters: `void main()`. §4.9, §17.10
- `instanceof` — Operator testing an object's run-time type; with a pattern it also binds a variable (Java 16). §3.5
- `Instant` — A point on the UTC time-line, stored as seconds and nanoseconds since the epoch; use it for timestamps. §7.10
- `int` — The 32-bit signed integer primitive, range -2^{31} … $2^{31}-1$. §2.3
- `Integer` **cache** — `Integer.valueOf` returns cached objects for −128 … 127, so `==` happens to work only in that range. §7.6
- **Integer division** — Dividing two integers drops the fraction, rounding toward zero: `7 / 2` is 3 and `-7 / 2` is −3. §2.8, §19.6
- **Integer overflow** — A result outside a type's range wraps around silently (`Integer.MAX_VALUE + 1` is `MIN_VALUE`). §2.10
- **Integration test** — A test that runs several real parts together (for example code plus a real database) instead of one class in isolation. §18.9
- **IntelliJ IDEA** — JetBrains' widely used Java IDE. §1.11
- **Interface** — A contract type: abstract, default, static and private methods plus constants; a class can implement many. §6.6
- **Intermediate operation** — A lazy stream operation such as `filter` or `map` that returns a new stream. §12.3
- **Interning** — Obtaining the pooled copy of a string with `intern()`. §7.1
- **Interpreter** — The JVM component that executes bytecode instruction by instruction before the JIT compiles hot code. §1.7
- **Interruption** — Cooperative cancellation: `interrupt()` sets a flag that blocking methods answer with `InterruptedException`. §14.4
- **Interval** — A range with a start and an end, such as a meeting `[9, 11)`; interval problems usually sort by start and sweep. §19.15
- **Intrinsic lock** — Another name for an object's *monitor*. §14.6
- **Invariance (generics)** — `List<Integer>` is not a `List<Number>` even though `Integer` is a `Number`; wildcards add the flexibility back. §9.6
- **Invariant** — A condition that must always hold — for an object after every public method, or for a loop on every iteration. §5.7
- **Inverse Ackermann function** — α(n), a function that grows so slowly it is below 5 for any realistic n; union-find costs O(α(n)) per operation. §19.18
- `invokedynamic` — A bytecode instruction whose call target is linked at run time; Java uses it for lambdas and string concatenation. §1.6
- **I/O** — Input/output: reading and writing files, network sockets, the console and databases. §13.1
- `IO` **class** — `java.lang.IO` (Java 25): `println`, `print` and `readln` for simple console programs. §4.9, §17.10
- `Iterable` — Anything with an `iterator()`; usable in an enhanced `for`. §10.1, §10.7
- `Iterator` — An object with `hasNext`, `next` and optional `remove` for walking a collection. §10.7

J

- **Jackson / Gson** — Popular third-party libraries for reading and writing JSON in Java. §13.12
- **Jakarta EE** — The Eclipse Foundation's enterprise Java specifications (formerly Java EE): Servlet, JPA, CDI, REST. §1.2, §18.14
- **JAR** — A ZIP archive of class files and resources with a manifest; executable with `java -jar` when it names a `Main-Class`. §18.12
- **Java agent** — A JAR loaded with `-javaagent` that can change classes as they load (used by profilers and Mockito); loading one into a running JVM now prints a warning. §17.6, §18.6

- **Java Memory Model (JMM)** — The rules defining when writes by one thread become visible to others. §15.8
- **Java SE** — Java Standard Edition: the core language, JVM and standard library this book covers. §1.2
- `javac` — The Java compiler; turns `.java` source into `.class` bytecode. §1.5
- **Javadoc** — Documentation comments `/** … */` and the tool that turns them into HTML API pages. §1.5, §2.1
- `JAVA_HOME` — The environment variable naming the JDK directory that build tools should use. §1.4
- `java.time` — The modern date-time API (Java 8): immutable, thread-safe classes such as `LocalDate` and `Instant`. §7.8
- **JCIP** — *Java Concurrency in Practice*, the classic book on Java threads and locking. §14.4
- `jcmd` — The command-line tool that sends diagnostic commands to a running JVM. §16.10
- **JDBC** — The standard API for talking to relational databases: `Connection`, `PreparedStatement`, `ResultSet`. §18.11
- **JDK** — Java Development Kit: the JVM, the class library and tools such as `javac`, `jar`, `jshell`, `jcmd`. §1.3
- **JDK Mission Control (JMC)** — A desktop application for analysing JFR recordings. §16.10
- **JEP** — JDK Enhancement Proposal: the numbered document describing a new feature. §1.12, §17.1
- **JFR (JDK Flight Recorder)** — Low-overhead event recording built into the JVM, for production profiling. §16.10
- **JIT compiler** — Just-in-time compiler that turns hot bytecode into optimised machine code while the program runs. §1.7, §16.8
- `jlink` — Builds a custom runtime image containing only the modules an application needs. §16.12
- **JLS** — The Java Language Specification: the official definition of the language's rules. §4.3
- **JMH** — The Java Microbenchmark Harness, which handles warm-up, forks and dead-code elimination for benchmarks. §16.9
- **JMX** — Java Management Extensions: the JVM's API for publishing and reading monitoring data and controls. §16.1
- **JNI** — Java Native Interface: the older way to call C or C++ code from Java, being replaced by the *Foreign Function & Memory API*. §16.1
- **Josephus problem** — n people stand in a circle and every k-th is removed; the survivor's position follows J(n) = (J(n − 1) + k) mod n. §19.27
- **JPA** — Jakarta Persistence: the standard API for mapping objects to database tables, usually through Hibernate. §1.2, §18.14
- **JPMS** — The Java Platform Module System (Java 9): `module-info.java` with `requires`, `exports`, `opens`. §16.12
- **JRE** — Java Runtime Environment: the JVM plus class library, enough to run but not compile programs; since Java 11 Oracle ships only the JDK (trimmed runtimes are built with `jlink`). §1.3
- `jshell` — The Java REPL (Java 9) for trying expressions interactively. §1.9
- **JSON** — JavaScript Object Notation: a common text format of objects, arrays, strings and numbers for exchanging data. §13.12
- `jstack` — Prints a thread dump of a running JVM. §16.10
- **JUL** — Short for `java.util.logging`, the JDK's built-in logging API. §18.10
- **JUnit** — The standard Java testing framework; tests are methods annotated `@Test`. §18.4
- **JUnit Platform / Jupiter / Vintage** — The parts of JUnit: the Platform discovers and launches tests, Jupiter is the modern test API, Vintage runs old JUnit 4 tests. §18.4
- **JVM** — Java Virtual Machine: loads, verifies and runs bytecode, manages memory and threads. §1.3, §16.1

K

- **Kafka** — A distributed, durable message log used to stream events between services. §20.18
- **Kahn's algorithm** — Topological sort by repeatedly taking a node with in-degree 0 and removing its outgoing edges. §19.17

- **Kernel** — The core of the operating system that manages memory, devices and scheduling; programs reach it through system calls. §16.13
- **Key derivation function API** — `javax.crypto.KDF` (Java 25) for deriving keys, e.g. with HKDF. §17.10
- **Keyword** — A word reserved by the language (`class`, `if`, `int`); contextual keywords such as `var` and `record` are special only in certain positions. §2.1
- **Kotlin** — A JetBrains language for the JVM that is fully interoperable with Java. §20.18
- **Kruskal's algorithm** — Building a minimum spanning tree by taking edges cheapest first and skipping any that would close a cycle (checked with union-find). §19.18

L

- **Label** — A name before a loop (`outer:`) that `break` or `continue` can target. §3.9
- **Lambda expression** — An anonymous function, `(params) -> body`, implementing a functional interface. §11.2
- **Latency** — The time one operation takes from start to finish — a request, a disk read, a GC pause. §16.5, §16.6, System Design Fundamentals
- **Lazy evaluation** — Deferring work until a result is needed; streams do nothing until a terminal operation runs. §12.2
- **LeetCode** — A popular website of coding-interview practice problems; "LeetCode-style" means a timed algorithm question. Ch 19
- **Lexicographic order** — Dictionary order: compare the first elements and look at the next ones only on a tie. §4.8
- **Lifecycle (Maven)** — Maven's fixed sequence of build phases (`validate` … `compile` … `test` … `package` … `deploy`); running a phase runs every earlier one. §18.2
- **Linked list** — A chain of nodes, each holding a value and a reference to the next node (and, if doubly linked, the previous one). §10.2, §19.22
- `LinkedHashMap` — A hash map that keeps insertion (or access) order; the basis of a simple LRU cache. §10.4
- `LinkedHashSet` — A hash set that remembers insertion order. §10.3
- `LinkedList` — A doubly linked `List` and `Deque`; O(1) at the ends, O(n) indexing. §10.2
- **Liskov substitution principle** — A subclass must work anywhere its parent is expected, without surprising the caller. §6.1
- `ListIterator` — An iterator over a `List` that can move in both directions and `set` or `add` elements as it goes. §10.2
- **Literal** — A value written directly in source code: `42`, `3.14`, `'a'`, `"hi"`, `true`. §2.4
- **Little's law** — Requests in flight = arrival rate × time per request; e.g. 1,000 requests/s each taking 0.1 s keep 100 busy at once. §15.4
- **Livelock** — Threads keep changing state in response to each other but make no progress. §15.9
- **Liveness** — A program's ability to keep making progress; deadlock, livelock and starvation are liveness failures. §15.9
- **Load average** — On Linux, the average number of threads running or waiting to run over the last 1, 5 and 15 minutes. §16.13
- **Load barrier** — A tiny piece of code ZGC adds to every reference read so it can move objects while the application keeps running. §16.5
- **Load factor** — The fullness ratio (default 0.75) at which a `HashMap` doubles its table. §10.5
- **Local class** — A class declared inside a method body. §6.11
- **Local variable type inference** — `var` (Java 10): the compiler infers a local variable's type from its initializer. §2.6
- `LocalDate` — A date without time or zone, e.g. `2025-03-09`. §7.9
- `LocalDateTime` — A date and time without a zone. §7.9
- **Locale** — A language-and-region setting (`Locale.US`, `Locale.ROOT`) that controls number, date and case formatting. §7.2, §7.4
- `LocalTime` / `OffsetDateTime` — `LocalTime` is a time of day with no date or zone; `OffsetDateTime` is a date-time with a fixed offset from UTC. §7.8

- **Lock elision** — The JIT removing locking on objects proved not to escape their thread. §16.8
- **Lock ordering** — Always acquiring several locks in one agreed global order, which makes deadlock impossible. §15.9
- **Logback / Log4j 2** — Logging back-ends that write SLF4J log calls to the console, files or other destinations. §18.10
- **Logging** — Recording events with a level and logger name to configurable destinations (SLF4J, Logback, `java.util.logging`). §18.10
- **Lombok** — A library that generates boilerplate such as getters at compile time by hooking into `javac`. §17.12
- **Lomuto partition** — A simple partition scheme that keeps one `store` index; everything left of it is smaller than the pivot. §19.25
- `long` — The 64-bit signed integer primitive, about ±9.2 × 10^{18}. §2.3
- `LongAdder` — A counter spread over several cells to avoid contention; faster than `AtomicLong` when many threads increment it. §14.10
- **Loop unrolling** — The JIT repeating a loop body several times per iteration to cut loop overhead. §16.8
- **Lowest common ancestor (LCA)** — The deepest tree node that has both given nodes beneath it (a node counts as its own descendant). §19.23
- **LRU cache** — A cache that evicts the least-recently-used entry; built with an access-ordered `LinkedHashMap`. §10.4
- **LTS release** — A long-term-support Java release (8, 11, 17, 21, 25) that vendors patch for years. §1.12

M

- `main` **method** — The program entry point: classically `public static void main(String[] args)`. §4.9
- **Manifest** — `META-INF/MANIFEST.MF` inside a JAR: metadata such as the `Main-Class` to run. §18.12
- `Map` — A key → value collection with unique keys; not a subtype of `Collection`. §10.4
- **Mark word** — The part of an object header that holds the identity hash code, lock state and GC age. §16.4
- **Mark-and-sweep** — GC that first marks every reachable object, then frees the rest. §16.5
- **Marker interface** — An interface with no methods that just tags a class, such as `Serializable`. §6.2
- `Math` — Utility class for arithmetic: `abs`, `pow`, `sqrt`, `round`, `floorMod`, `clamp`, `*Exact`. §7.5
- **Maven** — A build tool driven by `pom.xml`, with a fixed lifecycle of phases and a central repository. §18.2
- **Maven Central** — The main public repository from which Maven and Gradle download libraries. §18.1, §18.2
- **MDC (mapped diagnostic context)** — Per-thread key-value data, such as a request id, that the logging framework adds to every log line. §18.10
- **Megamorphic call** — A call site that has seen many different receiver classes, which the JIT can't inline well. §11.8, §16.8
- **Memoisation** — Caching a function's results by its arguments so each is computed only once — top-down dynamic programming. §19.19
- **Memory leak (Java)** — Objects that are no longer needed but stay reachable, so the GC can't free them. §16.11
- **Memory-mapped file** — A file mapped into the process's address space, so reading or writing memory reads or writes the file. §13.9
- **Merge sort** — Sorting by splitting the input in half, sorting each half and merging them; O(n log n) and stable. §19.25
- **Metaspace** — Native memory holding class metadata (replaced PermGen in Java 8). §16.3
- **Method** — A named block of code in a class that can take parameters and return a value. §4.1
- **Method overloading** — Several methods with the same name but different parameter lists, chosen at compile time. §4.3
- **Method overriding** — A subclass providing its own implementation of an inherited instance method, chosen at run time. §6.3

- **Method reference** — Shorthand for a lambda that calls one method: `String::length`, `System.out::println`, `ArrayList::new`. §11.4
- **Method signature** — A method's name plus its parameter types. §4.1
- **Microbenchmark** — A timing of a tiny piece of code; easy to get wrong without a harness such as JMH. §16.9
- **Min-heap / max-heap** — A heap whose root is always the smallest / largest element; `PriorityQueue` is a min-heap by default. §19.4, §19.13
- **Minimum spanning tree** — The cheapest set of edges that connects every node of a weighted graph without a cycle (Kruskal's algorithm). §19.18
- **Mock** — A test double that records calls and returns programmed answers (e.g. with Mockito). §18.6
- **Mockito** — The most widely used Java mocking library: `mock`, `when(...).thenReturn`, `verify`. §18.6
- **Modular arithmetic** — Working with remainders after division by a fixed number m; answers are often reported "mod 1,000,000,007" to keep them small. §19.6, §19.27
- **Modular JAR** — A JAR that contains a `module-info.class`. §18.12
- **Module** — A named, self-describing group of packages declared in `module-info.java`. §16.12
- **Module import declaration** — `import module java.base;` (Java 25): imports every package a module exports. §17.10
- **Module path** — Where the JVM looks for modules (`--module-path`), as opposed to the class path. §16.12
- **Mojibake** — Garbled text produced by decoding bytes with the wrong charset. §13.7
- **Monitor** — The lock every Java object has, used by `synchronized`, `wait` and `notify`. §14.6
- **Monotonic stack** — A stack kept in increasing or decreasing order by popping out-of-order items first; finds each "next greater element" in O(n). §19.12
- **Mounting / unmounting** — A virtual thread being attached to, or detached from, a carrier thread; blocking I/O unmounts it. §15.4
- **Multi-catch** — `catch (A \| B e)` — one handler for several unrelated exception types. §8.5
- **Multi-release JAR** — A JAR that holds different versions of some classes for different Java releases. §18.12
- **Mutable** — Able to change state after creation. §5.9
- **Mutable reduction** — Accumulating stream elements into a mutable container such as a list or map — what `collect` does. §12.5
- **Mutation testing** — Judging tests by planting small bugs ("mutants") and checking that the tests fail. §18.9
- **Mutual exclusion** — Letting only one thread at a time into a critical section. §14.6

N

- **NaN** — "Not a Number": the `double` result of undefined operations such as `0.0 / 0`; it is not even equal to itself. §2.11
- **Narrowing conversion** — Converting to a type with a smaller range (`long` → `int`); requires a cast and may lose information. §2.7
- **Native Image** — GraalVM's ahead-of-time compiler that turns a Java application into a fast-starting stand-alone executable. §18.12, §18.14
- **Native memory** — Memory the JVM uses outside the Java heap: metaspace, thread stacks, direct buffers, GC data. §16.3
- **Native Memory Tracking (NMT)** — A JVM option (`-XX:NativeMemoryTracking=summary`) that reports native memory use by area. §16.3
- **Natural order** — The order a class defines for itself by implementing `Comparable` (numbers ascending, strings alphabetically). §10.8
- **Nested class** — A class declared inside another; static nested or inner. §6.10
- **Netty** — A high-performance asynchronous networking framework for the JVM. §13.9
- **NIO.2** — The `java.nio.file` API (Java 7): `Path`, `Files`, `FileSystem`, `WatchService`. §13.1
- **Node** — One element of a linked structure (list, tree or graph) holding a value and references to its neighbours. §19.16, §19.22

- **Non-blocking** — An operation that returns immediately instead of making the thread wait, such as a CAS update. §14.10
- **NPE** — Short for `NullPointerException`. §8.1
- `null` — The reference value meaning "no object". §2.5
- `NullPointerException` — Thrown when code uses a `null` reference as if it pointed to an object. §8.1

O

- **Object** — An instance of a class, with state (fields) and behaviour (methods), living on the heap. §5.1
- `Object` **class** — The root of every class hierarchy; defines `equals`, `hashCode`, `toString`, `getClass`, `wait`, `notify`. §6.2, §6.13
- **Object header** — The bookkeeping at the start of every object — mark word and class pointer: 12 bytes, or 8 with compact headers. §16.4
- **Observer / listener** — An object registered to be told about events; forgetting to unregister listeners is a classic memory leak. §16.11
- **OCP** — Oracle Certified Professional: the Java SE 21 Developer certification exam (1Z0-830). §20.18
- **Off-by-one error** — A loop or index that goes one step too far or stops one too short (`<=` instead of `<`). §3.10
- **Old generation** — The heap area for objects that survived several young collections. §16.5
- **On-stack replacement (OSR)** — Switching a long-running loop from interpreted to compiled code while it is still running. §16.8
- **OOM** — Short for `OutOfMemoryError`. §16.3
- **OOP** — Object-oriented programming: organising code as objects that combine state and behaviour, using encapsulation, inheritance and polymorphism. §5.1
- **OpenJDK** — The open-source project and code base from which every mainstream Java distribution is built. §1.2
- **Operating system (OS)** — The software (Linux, Windows, macOS) that manages hardware, processes, memory and files for programs. §16.13
- **Operator** — A symbol that computes a value from operands, such as `+`, `==`, `&&` or `?:`. §2.8
- **Optimal substructure** — The best answer is built from the best answers to subproblems — one of the two conditions for dynamic programming. §19.19
- `Optional` — A container holding one value or none; a return type for "maybe no result". §11.7
- **Ordinal** — An enum constant's zero-based position in its declaration (`ordinal()`). §6.9
- `OutOfMemoryError` — Thrown when the JVM can't allocate memory in the heap, metaspace or native memory. §16.3
- **Overlapping subproblems** — Plain recursion would solve the same subproblem many times — the other condition for dynamic programming. §19.19

P

- **P99** — The 99th-percentile latency: 99 % of operations finished at least this fast. §16.6
- **Package** — A namespace grouping related types, declared with `package`; mirrors the directory structure. §5.4
- **Package-private** — The default access (no modifier): visible within the same package only. §5.6
- **Page cache** — Operating-system memory holding recently read or written file data, so repeated reads skip the disk. §16.13
- **Page fault** — A CPU trap when a program touches memory that isn't currently mapped to RAM; the OS must load or allocate the page. §16.13
- **Palindrome** — Text (or a list) that reads the same forwards and backwards. §19.8
- **Parallel GC** — A throughput collector that uses many threads in stop-the-world pauses. §16.5
- **Parallel stream** — A stream whose work is split across the common `ForkJoinPool`; only worth it for large CPU-bound tasks. §12.8
- **Parallelism** — Tasks literally running at the same instant on different CPU cores; concurrency only requires them to overlap in time. §14.1
- **Parameterized test** — A test run once per set of arguments (`@ParameterizedTest` with a source). §18.5

- **Partition** — Rearranging an array around a pivot so smaller values end up on its left; the core step of quicksort and quickselect. §19.25
- **Pass-by-value** — Java always copies the argument value into the parameter; for objects, the value copied is the reference. §4.2
- `Path` — An NIO.2 object naming a file-system location; doesn't touch the disk by itself. §13.2
- **Path compression** — In union-find, pointing nodes at (or nearer to) the root during `find`, so later finds are almost O(1). §19.18
- **Path traversal** — An attack that uses `../` in a user-supplied file name to escape the intended directory. §13.2
- **Pattern matching** — Testing a value's shape and extracting parts in one step: type, record and unnamed patterns. §3.5, §3.6, §3.7
- **Pattern variable** — The variable a pattern binds when it matches, such as `s` in `o instanceof String s`. §3.5
- **PECS** — "Producer `extends`, consumer `super`": the wildcard rule for flexible generic APIs. §9.6
- `Period` — A `java.time` amount in years, months and days (e.g. `P1Y2M5D`). §7.9
- **Permitted subclass** — A class listed in a sealed type's `permits` clause. §6.7
- `Phaser` — A reusable barrier-like synchronizer whose number of parties can change between phases. §14.13
- **Pinning** — A virtual thread blocking without unmounting, holding its carrier; `synchronized` no longer pins since Java 24. §15.5
- **Pivot** — The value a partition splits around; picking it at random avoids quicksort's O(n²) worst case. §19.25
- **Platform thread** — A Java thread backed one-to-one by an OS thread. §15.4
- **Plugin (build)** — An add-on that gives Maven or Gradle a job to do, such as compiling, running tests or packaging. §18.2
- **Pointer (in algorithms)** — An index or reference that marks a position while an algorithm walks the data, such as `left` and `right`. §19.8
- **Poison pill** — A special item put on a queue to tell its consumer to stop. §14.8, §14.12
- **Polymorphism** — One reference type, many run-time behaviours: a call runs the implementation of the object's actual class. §6.4
- **Precedence** — The order in which operators bind (`*` before `+`, `&&` before `\|\|`). §2.9
- **Precise rethrow** — `catch (Exception e) { throw e; }` keeps the narrower checked types the `try` can actually throw. §8.5
- **Prefix sum** — Running totals where `prefix[i]` is the sum of the first i elements, so any range sum costs one subtraction. §19.11
- `PreparedStatement` — A pre-compiled JDBC statement with `?` parameters; prevents SQL injection. §18.11
- **Preview feature** — A complete feature that may still change; needs `--enable-preview` on `javac` and `java`. §1.12, §17.1
- **Primitive stream** — `IntStream`, `LongStream` or `DoubleStream`, which avoid boxing and add `sum`/`average`. §12.6
- **Primitive type** — One of the eight built-in value types: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. §2.3
- `PriorityQueue` — A binary-heap queue that always removes the smallest element (by natural order or a comparator). §10.6
- `private` — Access modifier: visible only inside the declaring top-level class. §5.6
- **Process** — A running program with its own memory; the threads inside it share that memory. §14.1, §16.13
- **Producer–consumer** — One set of threads creating work items and another processing them, joined by a shared (usually blocking) queue. §14.12
- **Profiling** — Measuring where a running program spends CPU time or allocates memory (JFR, async-profiler). §16.10
- **Promotion (numeric)** — Automatic widening of `byte`, `short` and `char` to `int` (or to the wider operand type) in arithmetic. §2.7

- **`Properties`** — A key=value configuration map loaded from `.properties` files. §13.11
- **`protected`** — Access modifier: visible in the package and to subclasses. §5.6
- **Pseudo-random number generator** — An algorithm whose output looks random but is fully determined by its seed. §7.12
- **Pure function** — A function whose result depends only on its arguments and that has no side effects. §11.8

Q

- **Quarkus / Micronaut** — Cloud-native Java frameworks designed for fast start-up and low memory. §18.14, §20.18
- **Queue** — A collection that hands elements out in arrival order (FIFO): `offer` adds at the tail, `poll` removes from the head. §10.6
- **Quickselect** — Finding the k-th smallest element by partitioning and continuing on one side only; O(n) on average. §19.25
- **Quicksort** — Sorting by partitioning around a pivot and recursing on both sides; O(n log n) on average. §19.25

R

- **Race condition** — A result that depends on the timing of threads, e.g. lost updates from `count++`. §14.5
- **Raw type** — A generic type used without type arguments (`List`); disables generic type checking. §9.7
- **Reachability** — An object is live if a chain of references leads to it from a GC root. §16.5
- **Read-modify-write** — Reading a value, computing a new one and writing it back (`count++`); not atomic unless protected. §14.5
- **`ReadWriteLock`** — A lock pair allowing many concurrent readers or one writer. §14.9
- **Record** — A concise immutable data carrier (Java 16) with generated constructor, accessors, `equals`, `hashCode`, `toString`. §6.8
- **Record pattern** — A pattern that deconstructs a record: `case Point(int x, int y)` (Java 21). §3.7
- **Recurrence** — A formula that defines each answer from earlier ones, such as J(n) = (J(n − 1) + k) mod n. §19.19, §19.27
- **Recursion** — A method calling itself; needs a base case to stop. §4.5
- **Red-black tree** — A self-balancing binary search tree that keeps every operation O(log n); `TreeMap` and `TreeSet` use one. §10.4
- **Reduction** — Combining stream elements into one result with `reduce` or a collector. §12.4
- **Reentrant** — A lock its holder can acquire again without blocking itself, as `synchronized` and `ReentrantLock` allow. §14.6
- **`ReentrantLock`** — An explicit reentrant lock with `tryLock`, timeouts, fairness and conditions. §14.9
- **Refactoring** — Restructuring code to make it clearer without changing what it does; tests make it safe. §18.4, §18.9
- **Reference type** — Any non-primitive type; variables hold references to objects (or `null`). §2.5
- **Reflection** — Inspecting and using classes, methods and fields at run time (`getClass()`, `java.lang.reflect`). §16.2
- **Region (G1)** — One of the equal-sized blocks G1 divides the heap into; each region is eden, survivor, old or humongous at any time. §16.5
- **Regular expression** — A pattern language for matching text, used by `split`, `replaceAll`, `matches`, `Pattern`. §7.2
- **Reifiable type** — A type fully known at run time (`String`, `List<?>`, `int[]`); `List<String>` is not, because of erasure. §9.8
- **Rejection policy** — What a `ThreadPoolExecutor` does with a task when its queue is full and no thread is free — throw, run it in the caller, or drop it. §14.11
- **Relative / absolute path** — An absolute path starts at the file-system root; a relative path is resolved against the current directory. §13.2
- **REPL** — Read-eval-print loop: an interactive prompt such as `jshell`. §1.9

- **Resident set size (RSS)** — The physical memory a process actually occupies, as reported by the operating system. §16.10, §16.13
- **Resource leak** — Failing to close a file, socket or connection; prevented by try-with-resources. §8.6, §13.13
- **REST API** — A web API in which URLs name resources and HTTP methods (GET, POST …) act on them, usually exchanging JSON. §18.14
- **Reverse Polish Notation** — Postfix notation in which each operator follows its operands (`3 4 +`), evaluated with a stack. §19.12
- **RMI** — Remote Method Invocation: an old Java API for calling methods in another JVM, built on serialization. §13.10
- `Runnable` — A task interface with `void run()` and no result. §14.2
- `RuntimeException` — The superclass of unchecked exceptions. §8.2

S

- **Safepoint** — A point at which all Java threads can be paused safely, for example for parts of a garbage collection. §16.5
- **Scalar replacement** — Replacing a non-escaping object's fields with local variables so it's never allocated. §16.8
- **Scale (`BigDecimal`)** — The number of digits after the decimal point; `2.0` and `2.00` have different scales. §7.7
- `Scanner` — A `java.util` class that reads and parses console or file input token by token. §2.13
- `ScheduledExecutorService` — An executor that runs tasks after a delay or repeatedly at a fixed rate. §14.11
- **Scheduler** — The part of the OS (or, for virtual threads, of the JVM) that decides which thread runs on which core next. §15.4, §16.13
- **Scoped value** — An immutable value bound for the duration of a call and readable by code it invokes (final in Java 25). §15.6
- **SDKMAN!** — A command-line tool for installing and switching between JDK versions. §1.4
- **Sealed class** — A class or interface (Java 17) that lists its permitted subclasses with `permits`. §6.7
- `SecureRandom` — A random-number generator strong enough for passwords, keys and tokens. §7.12
- **Seed** — The starting value of a pseudo-random generator; the same seed reproduces the same sequence. §7.12
- **Selector** — The value a `switch` examines. §3.2
- `Semaphore` — A synchronizer handing out a fixed number of permits to limit concurrent access. §14.13
- **Sentinel** — A special marker value (such as −1 for "not seen") or dummy element that removes a special case from the code. §19.11, §19.22
- **Sequenced collection** — Java 21 interfaces (`SequencedCollection`, `SequencedSet`, `SequencedMap`) with `getFirst`, `getLast`, `reversed`. §10.11
- **Serial GC** — A single-threaded, stop-the-world collector for small heaps. §16.5
- **Serialization** — Converting objects to bytes with `ObjectOutputStream`; risky with untrusted data. §13.10
- `serialVersionUID` — The version number checked when deserialising a `Serializable` class. §13.10
- `Set` — A collection with no duplicate elements. §10.3
- **Shadowing** — A local variable or parameter hiding a field with the same name; use `this.name` to reach the field. §5.2
- **Shallow immutability** — Fields can't be reassigned, but the objects they refer to may still be mutable (true of records). §5.9, §6.8
- **Shebang** — A first line such as `#!/usr/bin/java --source 25` that lets a Java source file run as a Unix script. §1.10
- **Shenandoah** — A low-pause OpenJDK collector that compacts concurrently. §16.5
- **Short-circuit evaluation** — `&&` and `\|\|` skip the right operand when the left decides the result; also short-circuiting stream operations. §2.8, §12.4

- **Side effect** — Anything a function does besides returning a value: changing a field, printing, writing a file. §11.8
- **Sieve of Eratosthenes** — Finding every prime up to n by crossing out the multiples of each prime in turn. §19.27
- **SIGKILL** — The Unix signal that ends a process at once, with no chance to clean up (sent, for example, by the OOM killer). §16.13
- **SIMD** — Single instruction, multiple data: one CPU instruction that works on several numbers at once. §16.8
- **Single-file source launch** — Running `java Hello.java` without a separate compile step (Java 11). §1.10
- **Singleton** — A class with exactly one instance, often written as an `enum` with one constant. §5.3
- **SLF4J** — Simple Logging Facade for Java: a logging API with pluggable backends such as Logback. §18.10
- **Sliding window** — Keeping a running summary of a contiguous range while moving its ends forward, so subarray problems take O(n). §19.9
- **SNAPSHOT version** — A Maven version ending in `-SNAPSHOT`, meaning "work in progress — may change". §18.1
- **SOLID** — Five object-oriented design principles: single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion. §20.18
- **Space complexity** — How much extra memory an algorithm needs as its input grows, in Big-O terms. §19.7
- `Spliterator` — The splittable iterator behind every stream, which divides the work for parallel streams. §12.2
- **Spring Boot** — The most widely used Java framework for back-end services, with auto-configuration and starters. §18.14
- **Spy** — A test double that wraps a real object and records calls, letting you stub only some methods. §18.6
- **SQL** — Structured Query Language: the language for querying and changing relational databases. §18.11
- **SQL injection** — An attack in which user input is pasted into SQL text and runs as commands; `PreparedStatement` parameters prevent it. §18.11
- **Stable sort** — A sort that keeps equal elements in their original relative order. §10.8, §19.5
- **Stack (data structure)** — A LIFO collection: `push` adds on top and `pop` removes from the top; use `ArrayDeque`, not `Stack`. §10.6, §19.4
- **Stack (JVM)** — Per-thread memory of frames for active method calls, holding locals and partial results. §16.3
- **Stack trace** — The list of method frames active when an exception was created, newest first. §8.9
- **Stack unwinding** — An exception travelling up the call stack, abandoning each frame until a matching `catch` is found. §8.1
- `StackOverflowError` — Thrown when a thread's stack is exhausted, usually by runaway recursion. §4.5, §16.3
- `StampedLock` — A lock with an optimistic-read mode for read-mostly data; not reentrant. §14.9
- **Standard streams** — `System.in`, `System.out` and `System.err`: the input, output and error channels every program starts with. §13.8
- **Starvation** — A thread never gets the CPU or lock it needs because others always win. §15.9
- **Stateless / stateful operation** — A stateless stream step (`map`, `filter`) looks at one element at a time; a stateful one (`sorted`, `distinct`) must remember others. §12.3
- `static` — Belonging to the class rather than to an instance; one copy shared by all. §5.5
- **Static nested class** — A nested class declared `static`; it has no enclosing instance. §6.10
- **Stop-the-world pause** — A GC phase during which all application threads are paused. §16.5
- **Stream** — A lazy, one-shot pipeline of operations over a source of elements (`java.util.stream`). §12.1
- **Stream gatherers** — See *gatherer*. §12.9
- `String` — An immutable sequence of UTF-16 characters. §2.12, §7.1
- **String pool** — The JVM table of shared `String` objects for literals and interned strings. §7.1

- **String templates** — A preview feature in Java 21 and 22 that was withdrawn in Java 23. §17.8
- `StringBuilder` — A mutable, unsynchronised character buffer for building strings efficiently. §7.3
- **Strong encapsulation** — Since Java 17 JDK internals are inaccessible by default; `--add-opens` is required to reach them. §17.3
- **Structured concurrency** — Treating a group of concurrent subtasks as one unit (`StructuredTaskScope`); still preview in Java 25. §15.7
- **Structured logging** — Logging events as named fields (often JSON) that machines can search, rather than only free text. §18.10
- **Stub** — A test double that returns canned answers. §18.6
- **STW** — Short for *stop-the-world pause*. §16.5
- **Subarray / subsequence** — A subarray (or substring) is a contiguous slice; a subsequence keeps the original order but may skip elements. §19.9, §19.19
- `super` — Refers to the parent class: `super.method()` calls the parent's version, `super(...)` its constructor. §6.1
- **Suppressed exception** — An exception from `close()` attached to the primary exception in try-with-resources. §8.6
- **Surefire** — The Maven plugin that runs unit tests (its sibling Failsafe runs integration tests). §18.2
- **Surrogate pair** — Two `char`s encoding one Unicode character outside the Basic Multilingual Plane (many emoji). §7.2
- **Survivor space** — Young-generation areas to which objects that survived a collection are copied before promotion to the old generation. §16.5
- `switch` **expression** — A `switch` that yields a value (Java 14), using arrow labels or `yield`. §3.3
- `synchronized` — Keyword that acquires an object's monitor for a method or block, giving mutual exclusion and visibility. §14.6
- **Synchronizer** — A coordination object such as a latch, barrier, semaphore or phaser. §14.13
- **System call** — A request from a program to the operating-system kernel, such as reading a file. §16.13

T

- **Tabulation** — Bottom-up dynamic programming: filling a table in an order where every entry's dependencies are already computed. §19.19
- **Template Method pattern** — A base class fixes the steps of an algorithm and lets subclasses fill in some of them. §6.5
- `TemporalAdjusters` — Ready-made date adjustments such as "last day of the month" or "next Monday". §7.9
- **Terminal operation** — The stream operation that produces a result and triggers execution (`collect`, `count`, `forEach`). §12.4
- **Ternary operator** — `cond ? a : b` — the conditional expression. §2.8
- **Test double** — Any stand-in for a real collaborator in a test: dummy, stub, fake, spy or mock. §18.6
- **Testcontainers** — A library that starts real services (databases, message brokers) in throwaway Docker containers for tests. §18.9
- **Test-driven development (TDD)** — Writing a failing test first, then the code to make it pass, then tidying up. §18.9
- **Text block** — A multi-line string literal delimited by `"""` (Java 15). §2.12
- `this` — A reference to the current object; `this(...)` calls another constructor. §5.2, §5.3
- **Thread** — An independent path of execution within a process, sharing the heap with other threads. §14.1
- **Thread confinement** — Keeping data reachable by only one thread, so it needs no locking. §14.14
- **Thread dump** — A snapshot of every thread's state and stack; shows deadlocks. §16.10
- **Thread pool** — A set of reusable worker threads that execute submitted tasks. §14.11
- **Thread priority** — A hint from 1 to 10 for the scheduler; the operating system may ignore it. §15.9
- `ThreadLocal` — A variable with an independent value per thread. §15.6

- **Thread-per-request** — A server model that gives every request its own thread; affordable at scale with virtual threads. §15.4
- **`ThreadPoolExecutor`** — The configurable thread-pool class behind most `Executors` factory methods. §14.11
- **Thread-safe** — Behaving correctly when used by several threads at once without extra coordination by callers. §14.1, §14.14
- **Throughput** — Work completed per unit of time — requests per second, or for a GC the share of time not spent collecting. §16.5, System Design Fundamentals
- **`throw` / `throws`** — `throw` raises an exception; `throws` in a method signature declares checked exceptions it may raise. §8.3, §8.7
- **`Throwable`** — The root class of everything that can be thrown: `Error` and `Exception`. §8.2
- **Tiered compilation** — Running code in the interpreter, then C1, then C2 as it gets hotter. §1.7
- **Time slice** — The short period a thread runs before the scheduler may switch the core to another thread. §16.13
- **Time zone** — A region's rules for its offset from UTC, including daylight saving, such as `Europe/Paris`. §7.10
- **TimSort** — The stable merge sort used by `Arrays.sort` and `List.sort` for objects; very fast on partly sorted data. §19.5
- **TLAB** — Thread-local allocation buffer: a private chunk of eden from which one thread allocates without locking. §16.5
- **TLB** — Translation lookaside buffer: the CPU's cache of virtual-to-physical address translations. §16.13
- **TLS** — Transport Layer Security: the encryption behind HTTPS. §17.10
- **Toolchain** — A build-tool setting that picks the exact JDK used to compile and test, independent of the JDK running the build. §18.3
- **Topological sort** — Ordering a DAG's nodes so every edge points forward — prerequisites before the tasks that need them. §19.17
- **`toString`** — Returns a text representation; `Object`'s default is `ClassName@hash`. §6.13
- **Transaction** — A group of database changes that succeed or fail together (`commit` or `rollback`). §18.11
- **Transitive dependency** — A library you get indirectly because one of your dependencies needs it. §18.1, §18.2
- **Treeification** — `HashMap` turning a bucket of more than 8 entries into a small red-black tree (Java 8+, once the table has ≥ 64 buckets). §10.5
- **`TreeMap` / `TreeSet`** — Red-black-tree map/set kept in sorted order, O(log n), with navigation methods such as `floorKey`. §10.4, §10.3
- **Trie** — A prefix tree that stores strings character by character, so prefix lookups cost O(length of the prefix). §19.21
- **try-with-resources** — `try (Resource r = …)`, which closes every declared `AutoCloseable` automatically, in reverse order. §8.6
- **Two pointers** — Two indexes moving through data in a coordinated way, so one pass replaces a nested loop. §19.8
- **Two's complement** — How Java stores signed integers: the top bit is the sign and −x equals `~x + 1`. §2.10, §19.26
- **Type erasure** — Compiling generics by checking types and then removing type arguments, inserting casts. §9.7
- **Type inference** — The compiler deducing types (`var`, diamond, generic method arguments, lambda parameters). §2.6, §9.4
- **Type parameter** — A placeholder type such as `T` in `class Box<T>`. §9.2
- **Type pattern** — A pattern that tests a type and binds a variable: `instanceof String s`, `case Integer i`. §3.5
- **Type witness** — An explicit type argument written before a generic method's name, e.g. `Comparator.<int[]>comparingInt(...)`. §9.4, §19.5

U

- **Unboxing** — Converting a wrapper object back to its primitive; unboxing `null` throws `NullPointerException`. §7.6
- **Unchecked exception** — `RuntimeException`, `Error` and their subclasses — not checked by the compiler. §8.2
- **Unchecked warning** — A compiler warning that a generic operation can't be type-checked (raw types, unchecked casts). §9.8
- **`UncheckedIOException`** — An unchecked wrapper for `IOException`, used inside lambdas and streams. §13.4
- **Unicode** — The universal character set; Java strings use the UTF-16 encoding of it. §7.2
- **Union-find (disjoint set union)** — A structure that tracks which elements belong to the same group and merges groups in nearly O(1) per operation. §19.18
- **Unit test** — A fast test of one small piece of code — a method or a class — in isolation. §18.4, §18.9
- **Unmodifiable collection** — A collection whose mutators throw `UnsupportedOperationException`, such as `List.of`. §10.10
- **Unnamed module** — The module that holds everything loaded from the class path. §16.12
- **Unnamed variable `_`** — An underscore for a variable or pattern you don't use (Java 22). §3.7, §17.7
- **`UnsupportedOperationException`** — Thrown by an operation a collection or object doesn't support. §10.10
- **Upcasting** — See *downcasting / upcasting*. §6.4
- **UTC** — Coordinated Universal Time: the world reference time, with no time-zone or daylight-saving shifts. §7.10
- **UTF-16** — The encoding Java strings use: each character is one or two 16-bit `char`s. §2.3, §7.2
- **UTF-8** — A variable-length encoding using 1–4 bytes per character; the Java default charset since 18. §13.7

V

- **Value object** — A small immutable object defined only by its values (an amount of money, a date range); records fit well. §6.8, §18.6
- **`var`** — Local variable type inference (Java 10). §2.6
- **Varargs** — A parameter `T... values` accepting zero or more arguments as an array. §4.4
- **Vector API** — An incubating API for SIMD calculations — one instruction applied to many numbers at once. §17.11
- **Virtual memory** — Each process sees its own large address space, which the OS maps onto physical RAM page by page. §16.13
- **Virtual thread** — A lightweight thread (Java 21) scheduled by the JVM onto carrier threads; cheap enough to create one per task. §15.4
- **Visibility (memory)** — Whether one thread's writes are guaranteed to be seen by another. §14.7, §15.8
- **VisualVM / Eclipse MAT / async-profiler** — Free diagnostic tools: VisualVM monitors a running JVM, Eclipse MAT analyses heap dumps, async-profiler samples CPU and allocations into flame graphs. §16.10
- **`volatile`** — A field modifier guaranteeing visibility and ordering of reads and writes, but not atomic compound actions. §14.7

W

- **`wait` / `notify`** — `Object` methods for guarded blocks; must be called holding the object's monitor. §14.8
- **Wait set** — The threads waiting in `wait()` on an object's monitor until they are notified. §14.8
- **Warm-up** — The start-up period while the JVM loads classes and the JIT compiles hot code; timings taken then are misleadingly slow. §16.9
- **`WatchService`** — The NIO.2 API that reports files being created, changed or deleted in a directory. §13.9
- **Weakly consistent iterator** — A concurrent-collection iterator that never throws CME and may or may not show concurrent updates. §14.12

- **Week-based year** — `YYYY` in a date pattern: the year a week belongs to, which differs from the calendar year `yyyy` around New Year. §7.11
- **Widening conversion** — Automatic conversion to a type with a larger range (`int` → `long` → `double`). §2.7
- **Wildcard** — `?` in a generic type: `List<?>`, `List<? extends T>`, `List<? super T>`. §9.6
- **Work stealing** — Idle threads taking tasks from the back of busy threads' queues, as `ForkJoinPool` does. §15.3
- **Wrapper class** — The object form of a primitive: `Integer`, `Long`, `Double`, `Character`, `Boolean`, … §7.6

X

- **XML** — Extensible Markup Language: a tag-based text format, used for example by Maven's `pom.xml`. §18.2

Y

- `yield` — Returns a value from a block inside a `switch` expression. §3.3
- **Young generation** — The heap area (eden plus survivor spaces) where new objects are allocated; collected often and cheaply. §16.5

Z

- **Zero-copy** — Moving data between files and sockets without copying it through the Java heap (e.g. `transferTo`). §13.9
- **ZGC** — A scalable low-latency collector with sub-millisecond pauses; generational-only since Java 24. §16.5
- `ZonedDateTime` — A date-time with a time-zone region, aware of daylight-saving rules. §7.10

Chapter TL;DR

- **Types**: eight primitives with fixed sizes (`int` ±2.1 × 10^9, `long` ±9.2 × 10^{18}, `char` 0–65 535); integer overflow wraps silently; `0.1 + 0.2 != 0.3`; money in `BigDecimal` built from strings.
- **Equality**: `==` compares references for objects; use `equals` — the `Integer` cache (−128…127) and the string pool only make `==` *look* right.
- **Modern syntax**: switch expressions, pattern matching with guards, record patterns and `_`, sealed types for exhaustive switches, records for data, `var` for locals, text blocks.
- **Collections**: `ArrayList` and `HashMap` by default, `ArrayDeque` for stacks and queues, `TreeMap` for order and ranges, `LinkedHashMap` for LRU; `List.of` is unmodifiable and rejects `null`.
- **Functional and streams**: `Supplier`/`Consumer`/`Predicate`/`Function`; streams are lazy and one-shot; `groupingBy`, `partitioningBy`, `toMap` (duplicates throw); gatherers in 24; `Optional` only as a return type.
- **Exceptions**: checked = recoverable and declared, unchecked = bugs; try-with-resources closes in reverse and keeps close failures as suppressed; never `return` from `finally`.
- **Concurrency**: `synchronized`/locks for mutual exclusion, `volatile` for visibility only, atomics and `ConcurrentHashMap` for counters and caches, `CompletableFuture` for composition, virtual threads for blocking I/O at scale; happens-before decides visibility.
- **JVM**: heap, stacks, metaspace, code cache; G1 by default, ZGC for low latency; `-Xmx`, `-Xlog:gc*`, heap dumps on OOM; `jcmd`, JFR, `jstack` to diagnose.
- **Versions**: 17 sealed · 21 virtual threads, pattern `switch`, record patterns, sequenced collections · 22 `_`, FFM · 24 gatherers · 25 compact source files, module imports, flexible constructors, scoped values. Structured concurrency is still preview.
- **Tools**: `mvn clean verify` / `./gradlew build`, JUnit + Mockito + AssertJ, SLF4J for logging, `PreparedStatement` for SQL.
- **Next**: Spring Boot (or Jakarta EE, Quarkus), Kotlin if you like, and the LLD, SQL and DDIA notes — then build something real.